

프로젝트 주제는 자유
이런 거 해 볼 수 있을 거 같은데 기록해 두기.
시간이 얼마 없으니 주제를 미리 생각해 두는 게 좋을 듯.

데이터 전처리

- 데이터 집합 불러오기

> read_csv('경로/파일명', sep='\t', encoding='utf-8')

```
import pandas as pd
df = pd.read_csv('data/gapminder.tsv', sep='\t')
```

※ 판다스에서 사용하는 자료형 :

시리즈(Series)

데이터프레임(DataFrame)

Tuple 비슷한 거 같다.

0	25
1	5
2	5
3	15
4	2
dtype: int64	

서울	25
대전	5
광주	5
부산	15
제주	2
dtype: int64	

구 국번		
서울	25	02
대전	5	042
광주	5	062
부산	15	051
제주	2	064

• 데이터 집합 불러오기

> head(n)

- 데이터프레임의 데이터를 확인하는 용도로 사용
- 가장 앞에 있는 5개의 행을 출력 (n 숫자 지정 시 개수만큼 출력)

```
df.head()
```

Out :

	country	continent	year	lifeExp	pop	gdpPercap
0	Afghanistan	Asia	1952	28.801	8425333	779.445314
1	Afghanistan	Asia	1957	30.332	9240934	820.853030
2	Afghanistan	Asia	1962	31.997	10267083	853.100710
3	Afghanistan	Asia	1967	34.020	11537966	836.197138
4	Afghanistan	Asia	1972	36.088	13079460	739.981106

- 데이터 집합 불러오기

> `type()` – 자료형 확인

```
type(df)
```

Out : `pandas.core.frame.DataFrame`

> `shape` – 데이터의 행과 열의 크기 정보

```
df.shape
```

Out : `(1704, 6)`

> `columns` – 데이터프레임의 열 이름 정보

```
df.columns
```

Out : `Index(['country', 'continent', 'year', 'lifeExp', 'pop',
 'gdpPercap'], dtype='object')`

• 데이터 집합 불러오기

> dtypes – 데이터프레임의 모든 열 자료형 확인

```
df.dtypes
```

```
Out : country      object
      continent    object
      year         int64
      lifeExp      float64
      pop          int64
      gdpPercap    float64
      dtype: object
```

> info() – 데이터프레임의 상세 정보 확인

```
df.info()
```

[.type](#), [.shape](#), [columns](#), + [nullcheck](#)

```
Out : <class 'pandas.core.frame.DataFrame'>
      RangeIndex: 1704 entries, 0 to 1703
      Data columns (total 6 columns):
      #   Column      Non-Null Count  Dtype
      ---  ---
      0   country    1704 non-null    object
      1   continent  1704 non-null    object
      2   year       1704 non-null    int64
      3   lifeExp    1704 non-null    float64
      4   pop        1704 non-null    int64
      5   gdpPercap  1704 non-null    float64
      dtypes: float64(2), int64(2), object(2)
      memory usage: 80.0+ KB
      None
```

데이터분석에서는 메모리 사용량이 중요함.

• 데이터 집합 불러오기

> 판다스와 파이썬 자료형 비교

판다스 자료형	파이썬 자료형	설명
str object	string	문자열
int64	int	정수
float64	float	소수점을 가진 숫자
datetime64	datetime	파이썬 표준 라이브러리인 datetime이 반환하는 자료형

int64 데이터의 크기
64bit로 표현가능한 숫자의 범위.
 $2^{64} = -2^{63} \sim (2^{63}) - 1$

1byte=8bits

자료형을 구분해 놓은 이유:
최대한 메모리(RAM) 절약하기 위함.
특히 작은 숫자는 작은 크기 자료형으로 사용하기.

시계열데이터 분석은 지금 과정에선 하지 않는다.

• 데이터 추출하기

> df['열 이름']

```
country_df = df['country']  
type(country_df)
```

Out : pandas.core.series.Series

> **tail(n)**

- 데이터프레임의 데이터를 확인하는 용도로 사용
- 가장 뒤에 있는 5개의 행을 출력 (n 숫자 지정 시 개수만큼 출력)

```
country_df.tail()
```

Out :

1699	Zimbabwe
1700	Zimbabwe
1701	Zimbabwe
1702	Zimbabwe
1703	Zimbabwe

Name: country, dtype: object

- 데이터 추출하기

> 여러 개의 열 데이터 추출하기

```
subset=df[['country', 'continent', 'year']]  
type(subset)
```

Out : pandas.core.frame.DataFrame

```
subset.tail()
```

Out :

	country	continent	year
1699	Zimbabwe	Africa	1987
1700	Zimbabwe	Africa	1992
1701	Zimbabwe	Africa	1997
1702	Zimbabwe	Africa	2002
1703	Zimbabwe	Africa	2007

• 데이터 추출하기

> 행 단위 데이터 추출하기

- loc : 인덱스를 기준으로 행 데이터 추출
- iloc : 행 번호를 기준으로 행 데이터 추출

dataframe이 가지는 가장 왼쪽 column값, 인덱스를 넣어줘야함.

index 숫자가 아닌 문자, 예) country인 경우
문자 index 사용시.
Dictionary라고 생각하세요

행 번호, 순번 0,1,2를 넣어주면 됨.
리스트라고 생각하세요.

	country	continent	year	lifeExp	pop	gdpPercap
0	Afghanistan	Asia	1952	28.801	8425333	779.445314
1	Afghanistan	Asia	1957	30.332	9240934	820.853030
2	Afghanistan	Asia	1962	31.997	10267083	853.100710
3	Afghanistan	Asia	1967	34.020	11537966	836.197138
4	Afghanistan	Asia	1972	36.088	13079460	739.981106

인덱스

- > 인덱스는 보통 0부터 시작하지만, 특정 열을 지정하여 사용 가능
행 번호는 0부터 시작하는 정수형태의 데이터 순서

• 데이터 추출하기

함수가 아니라 attribute라 []를 사용한다.. 리스트 비슷한 친구인듯?

> loc 속성으로 행 데이터 추출하기

```
df.loc[0]
```

```
Out : country      Afghanistan
      continent     Asia
      year         1952
      lifeExp      28.801
      pop          8425333
      gdpPercap    779.445
      Name: 0, dtype: object
```

```
df.loc[99]
```

```
Out : country      Bangladesh
      continent     Asia
      year         1967
      lifeExp      43.453
      pop          62821884
      gdpPercap    721.186
      Name: 99, dtype: object
```

```
df.loc[-1]
```

```
-----
KeyError
1 last)
```

Traceback (most recent call

- 데이터 추출하기

> 마지막 행 데이터 추출하기

```
number_of_rows = df.shape[0]
last_row_index = number_of_rows - 1
df.loc[last_row_index]
```

Out :

country	Zimbabwe
continent	Africa
year	2007
lifeExp	43.487
pop	12311143
gdpPercap	469.709
Name: 1703, dtype: object	

```
df.tail(1)
```

Out :

	country	continent	year	lifeExp	pop	gdpPercap
1703	Zimbabwe	Africa	2007	43.487	12311143	469.709298

- 데이터 추출하기

> 여러 행 데이터 추출하기

```
df.loc[[0, 99, 999]]
```

Out :

	country	continent	year	lifeExp	pop	gdpPercap
0	Afghanistan	Asia	1952	28.801	8425333	779.445314
99	Bangladesh	Asia	1967	43.453	62821884	721.186086
999	Mongolia	Asia	1967	51.253	1149500	1226.041130

```
df.loc[0:2]
```

Out :

	country	continent	year	lifeExp	pop	gdpPercap
0	Afghanistan	Asia	1952	28.801	8425333	779.445314
1	Afghanistan	Asia	1957	30.332	9240934	820.853030
2	Afghanistan	Asia	1962	31.997	10267083	853.100710

- 데이터 추출하기

> `iloc` 속성으로 행 데이터 추출하기

```
df.iloc[0]
```

```
Out : country      Afghanistan  
       continent      Asia  
       year         1952  
       lifeExp      28.801  
       pop          8425333  
       gdpPercap    779.445  
       Name: 0, dtype: object
```

```
df.iloc[99]
```

```
Out : country      Bangladesh  
       continent      Asia  
       year         1967  
       lifeExp      43.453  
       pop          62821884  
       gdpPercap    721.186  
       Name: 99, dtype: object
```

- 데이터 추출하기

> iloc 속성으로 행 데이터 추출하기

```
df.iloc[-1]
```

```
Out : country      Zimbabwe  
      continent    Africa  
      year         2007  
      lifeExp      43.487  
      pop          12311143  
      gdpPercap    469.709  
      Name: 1703, dtype: object
```

```
df.iloc[1710]
```

```
-----  
-----  
IndexError                                Traceback (most recent call  
1 last)  
<ipython-input-28-e91d411323c7> in <module>()  
----> 1 print(df.iloc[1710])
```

• 데이터 추출하기

- > loc, iloc 속성 자유자재로 사용하기
 - 슬라이싱 구문으로 데이터 추출하기

```
subset = df.loc[:, ['year', 'pop']]  
subset.head()
```

Out :

	year	pop
0	1952	8425333
1	1957	9240934
2	1962	10267083
3	1967	11537966
4	1972	13079460

• 데이터 추출하기

- > loc, iloc 속성 자유자재로 사용하기
 - 슬라이싱 구문으로 데이터 추출하기

```
subset = df.iloc[:, [2, 4, -1]]  
subset.head()
```

Out :

	year	pop	gdpPercap
0	1952	8425333	779.445314
1	1957	9240934	820.853030
2	1962	10267083	853.100710
3	1967	11537966	836.197138
4	1972	13079460	739.981106

• 데이터 추출하기

- > loc, iloc 속성 자유자재로 사용하기
 - 슬라이싱 구문으로 데이터 추출하기

```
subset = df.iloc[:, range(5)]  
subset.head()
```

Out :

	country	continent	year	lifeExp	pop
0	Afghanistan	Asia	1952	28.801	8425333
1	Afghanistan	Asia	1957	30.332	9240934
2	Afghanistan	Asia	1962	31.997	10267083
3	Afghanistan	Asia	1967	34.020	11537966
4	Afghanistan	Asia	1972	36.088	13079460

• 데이터 추출하기

- > loc, iloc 속성 자유자재로 사용하기
 - 여러 개의 행, 열 데이터 추출하기

```
df.loc[[0, 99, 999], ['country', 'lifeExp', 'gdpPercap']]
```

Out :

	country	lifeExp	gdpPercap
0	Afghanistan	28.801	779.445314
99	Bangladesh	43.453	721.186086
999	Mongolia	51.253	1226.041130

```
df.iloc[[0, 99, 999], [0, 3, 5]]
```

Out :

	country	lifeExp	gdpPercap
0	Afghanistan	28.801	779.445314
99	Bangladesh	43.453	721.186086
999	Mongolia	51.253	1226.041130

• 기초적인 통계 계산하기

> 그룹화한 데이터의 평균 구하기

- `groupby('그룹화 열 이름')['대상 열'].mean()`

```
df.groupby('year')['lifeExp'].mean()
```

Out :

year	
1952	49.057620
1957	51.507401
1962	53.609249
1967	55.678290
1972	57.647386
1977	59.570157
1982	61.533197
1987	63.212613
1992	64.160338
1997	65.014676
2002	65.694923
2007	67.007423

Name: lifeExp, dtype: float64

- 기초적인 통계 계산하기

- > 그룹화한 데이터의 평균 구하기
 - groupby() 메소드 살펴보기

```
grouped_year_df = df.groupby('year')  
type(grouped_year_df)
```

Out : pandas.core.groupby.generic.DataFrameGroupBy

```
grouped_year_df_lifeExp = grouped_year_df['lifeExp']  
type(grouped_year_df_lifeExp)
```

Out : pandas.core.groupby.generic.SeriesGroupBy

• 기초적인 통계 계산하기

- > 그룹화한 데이터의 평균 구하기
 - groupby() 메소드 살펴보기

```
mean_lifeExp_by_year = grouped_year_df_lifeExp.mean()
mean_lifeExp_by_year.head(2)
```

```
Out :    year
      1952    49.057620
      1957    51.507401
      Name: lifeExp, dtype: float64
```

```
type(mean_lifeExp_by_year)
```

```
Out : pandas.core.series.Series
```

• 기초적인 통계 계산하기

> 그룹화한 데이터의 평균 구하기

- lifeExp, gdpPercap 열의 평균값을 연도, 지역별로 그룹화하여 계산

```
df.groupby(['year', 'continent'])[['lifeExp', 'gdpPercap']].mean()
```

Out :

		lifeExp	gdpPercap
year	continent		
1952	Africa	39.135500	1252.572466
	Americas	53.279840	4079.062552
	Asia	46.314394	5195.484004
	Europe	64.408500	5661.057435
	Oceania	69.255000	10298.085650
1957	Africa	41.266346	1385.236062
	Americas	55.960280	4616.043733
	Asia	49.318544	5787.732940
	Europe	66.703067	6963.012816
	Oceania	70.295000	11598.522455

• 기초적인 통계 계산하기

> 그룹화한 데이터의 개수 세기

- `nunique()` : 빈도수
- `continent`를 기준으로 그룹화한 후 `country` 개수 세기

```
df.groupby('continent')['country'].nunique()
```

Out :

continent	
Africa	52
Americas	25
Asia	33
Europe	30
Oceania	2

Name: country, dtype: int64

• 그래프 그리기

> 년도별 수명 데이터를 그룹화하여 그래프로 표현

- year 열을 기준으로 그룹화한 데이터프레임에서 lifeExp 열의 평균 계산

```
year_life_expectancy = df.groupby('year')['lifeExp'].mean()  
year_life_expectancy
```

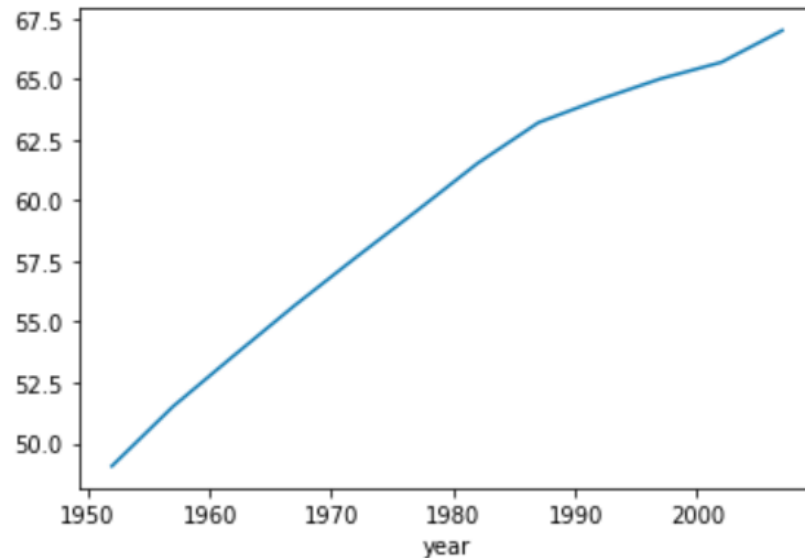
```
Out :   year  
      1952    49.057620  
      1957    51.507401  
      1962    53.609249  
      1967    55.678290  
      1972    57.647386  
      1977    59.570157  
      1982    61.533197  
      1987    63.212613  
      1992    64.160338  
      1997    65.014676  
      2002    65.694923  
      2007    67.007423  
      Name: lifeExp, dtype: float64
```


• 그래프 그리기

- > 년도별 수명 데이터를 그룹화하여 그래프로 표현
 - matplotlib 라이브러리 사용

```
%matplotlib inline  
import matplotlib.pyplot as plt  
year_life_expectancy.plot()
```

Out : <AxesSubplot: xlabel='year'>



• 나만의 데이터 만들기

> Series() 메소드에 리스트를 전달하여 시리즈 생성

```
s = pd.Series(['Wes McKinney','Creator of Pandas'])  
s
```

```
Out : 0      Wes McKinney  
      1  Creator of Pandas  
      dtype: object
```

> index 인자에 문자열을 리스트로 담아서 전달 가능

```
s = pd.Series(['Wes McKinney','Creator of Pandas'],  
              index=['Person','Who'])  
s
```

```
Out : Person      Wes McKinney  
      Who      Creator of Pandas  
      dtype: object
```

• 나만의 데이터 만들기

> 데이터프레임 만들기

- DataFrame() 메소드에 딕셔너리를 전달하여 데이터프레임 생성

```
scientists = pd.DataFrame({  
    'Name':['Rosaline Franklin','William Gosset'],  
    'Occupation':['Chemist','Statistician'],  
    'Born':['1920-07-25','1876-06-13'],  
    'Died':['1958-04-16','1937-10-16'],  
    'Age':[37,61]})  
scientists
```

Out :

	Name	Occupation	Born	Died	Age
0	Rosaline Franklin	Chemist	1920-07-25	1958-04-16	37
1	William Gosset	Statistician	1876-06-13	1937-10-16	61

• 나만의 데이터 만들기

> 데이터프레임 만들기

- DataFrame() 메소드에 딕셔너리를 전달하여 데이터프레임 생성

```
Scientists = pd.DataFrame(  
    data={'Occupation':['Chemist','Statistician'],  
          'Born':['1920-07-25','1876-06-13'],  
          'Died':['1958-04-16','1937-10-16'],  
          'Age':[37,61]},  
    index=['Rosaline Franklin','William Gosset'],  
    columns=['Occupation','Born','Age','Died'])  
  
scientists
```

Out :

	Occupation	Born	Age	Died
Rosaline Franklin	Chemist	1920-07-25	37	1958-04-16
William Gosset	Statistician	1876-06-13	61	1937-10-16

• 시리즈 다루기 - 기초

> 데이터프레임에서 시리즈 선택하기

```
first_row = scientists.loc['William Gosset']  
print(type(first_row))  
print(first_row)
```

Out : <class 'pandas.core.series.Series'>
Occupation Statistician
Born 1876-06-13
Age 61
Died 1937-10-16
Name: William Gosset, dtype: object

- 시리즈 다루기 - 기초

> index 속성 사용하기

```
first_row.index
```

Out : Index(['Occupation', 'Born', 'Age', 'Died'], dtype='object')

> values 속성 사용하기

```
first_row.values
```

Out : array(['Statistician', '1876-06-13', 61, '1937-10-16'], dtype=object)

- 시리즈 다루기 - 기초

> index 속성 응용하기

```
first_row.index[0]
```

Out : 'Occupation'

> values 속성 응용하기

```
first_row.values[0]
```

Out : 'Statistician'

- 시리즈 다루기 - 기초

- 컬럼명으로 추출하기

> scientists 데이터프레임의 Age 열 추출

```
ages = scientists['Age']  
ages
```

Out : Rosaline Franklin 37
William Gosset 61
Name: Age, dtype: int64

> scientists 데이터프레임의 Born, Died 열 추출

```
dates = scientists[['Born', 'Died']]  
dates
```

Out :

	Born	Died
Rosaline Franklin	1920-07-25	1958-04-16
William Gosset	1876-06-13	1937-10-16

- 시리즈 다루기 - 응용

> 시리즈, 데이터프레임 관련 메소드

시리즈 메소드	설명
isin	시리즈에 포함된 값이 있는지 확인
count	데이터 개수 반환
value_counts	데이터 별 개수 반환
min	최소값 반환
max	최대값 반환
mean	산술 평균 반환
median	중간값 반환
quantile	사분위 값(25%, 50%, 75%) 반환
describe	요약 통계량 계산
replace	특정 값을 가진 시리즈 값을 교체
sample	시리즈에서 임의의 값을 반환
sort_values	값을 정렬
to_frame	시리즈를 데이터프레임으로 변환

- 시리즈 다루기 - 응용

034

> 데이터로 함수 응용하기

```
scientists = pd.read_csv('data/scientists.csv')
ages = scientists['Age']
scientists
```

Out :

	Name	Born	Died	Age	Occupation
0	Rosaline Franklin	1920-07-25	1958-04-16	37	Chemist
1	William Gosset	1876-06-13	1937-10-16	61	Statistician
2	Florence Nightingale	1820-05-12	1910-08-13	90	Nurse
3	Marie Curie	1867-11-07	1934-07-04	66	Chemist
4	Rachel Carson	1907-05-27	1964-04-14	56	Biologist
5	John Snow	1813-03-15	1858-06-16	45	Physician
6	Alan Turing	1912-06-23	1954-06-07	41	Computer Scientist
7	Johann Gauss	1777-04-30	1855-02-23	77	Mathematician

• 시리즈 다루기 - 응용

- 데이터 포함여부 확인

> isin() 메소드 사용

```
scientists.isin([37, 41, 45])
```

Out :

	Name	Born	Died	Age	Occupation
0	False	False	False	True	False
1	False	False	False	False	False
2	False	False	False	False	False
3	False	False	False	False	False
4	False	False	False	False	False
5	False	False	False	True	False
6	False	False	False	True	False
7	False	False	False	False	False

• 시리즈 다루기 - 응용

- 데이터 개수 확인하기

> count() 메소드 사용

```
scientists.count()
```

Out :

```
Name      8
Born      8
Died      8
Age       8
Occupation 8
dtype: int64
```

count 보다는 info로 검토하는 것이 정확하다.

• 시리즈 다루기 - 응용

- 데이터 별 개수 확인하기

> value_counts() 메소드 사용

```
scientists['Occupation'].value_counts()
```

Out :

Chemist **2**

Statistician **1**

Nurse **1**

Biologist **1**

Physician **1**

Computer Scientist **1**

Mathematician **1**

Name: Occupation, dtype: int64

• 시리즈 다루기 - 응용

- 기초 통계 메소드 사용하기

> mean(), min(), max(), std() 메소드 사용

```
print(ages.mean())  
print(ages.min())  
print(ages.max())  
print(ages.quantile(q = 0.25))  
print(ages.median())
```

df는 여러 타입을 가지고 있으므로,
숫자로 된 series에만 사용.

Out : 59.125

37

90

44.0

58.5

- 시리즈 다루기 - 응용

- 기초 통계 메소드 사용하기

> describe() 메소드 사용

```
print(ages.describe())
```

Out :

count	8.000000
mean	59.125000
std	18.325918
min	37.000000
25%	44.000000
50%	58.500000
75%	68.750000
max	90.000000

Name: Age, dtype: float64

- 시리즈 다루기 - 응용

- 데이터 변경하기

> replace() 메소드 사용

```
scientists.replace(37, 40)
```

Out :

	Name	Born	Died	Age	Occupation
0	Rosaline Franklin	1920-07-25	1958-04-16	40	Chemist

> 인덱싱 사용

```
scientists.loc[0, 'Age'] = 40
```

잘 사용되지 않는다.

```
scientists
```

Out :

	Name	Born	Died	Age	Occupation
0	Rosaline Franklin	1920-07-25	1958-04-16	40	Chemist

• 시리즈 다루기 - 응용

- 데이터에서 표본 추출

> sample(n) 메소드 사용

```
scientists.sample(3)
```

Out :

	Name	Born	Died	Age	Occupation
2	Florence Nightingale	1820-05-12	1910-08-13	90	Nurse
6	Alan Turing	1912-06-23	1954-06-07	41	Computer Scientist
4	Rachel Carson	1907-05-27	1964-04-14	56	Biologist

> sample(frac) 메소드 사용

```
scientists.sample(frac=0.2)
```

Out :

	Name	Born	Died	Age	Occupation
1	William Gosset	1876-06-13	1937-10-16	61	Statistician
3	Marie Curie	1867-11-07	1934-07-04	66	Chemist

- 시리즈 다루기 - 응용

- 데이터 정렬

> sort_index() 메소드 사용

```
scientists.sort_index(ascending = True)
```

Out :

	Name	Born	Died	Age	Occupation
0	Rosaline Franklin	1920-07-25	1958-04-16	37	Chemist
1	William Gosset	1876-06-13	1937-10-16	61	Statistician
2	Florence Nightingale	1820-05-12	1910-08-13	90	Nurse
3	Marie Curie	1867-11-07	1934-07-04	66	Chemist
4	Rachel Carson	1907-05-27	1964-04-14	56	Biologist
5	John Snow	1813-03-15	1858-06-16	45	Physician
6	Alan Turing	1912-06-23	1954-06-07	41	Computer Scientist
7	Johann Gauss	1777-04-30	1855-02-23	77	Mathematician

• 시리즈 다루기 - 응용

- 데이터 정렬

> `sort_values()` 메소드 사용

```
scientists.sort_values('Age', ascending = True)
```

Out :

	Name	Born	Died	Age	Occupation
0	Rosaline Franklin	1920-07-25	1958-04-16	37	Chemist
6	Alan Turing	1912-06-23	1954-06-07	41	Computer Scientist
5	John Snow	1813-03-15	1858-06-16	45	Physician
4	Rachel Carson	1907-05-27	1964-04-14	56	Biologist
1	William Gosset	1876-06-13	1937-10-16	61	Statistician
3	Marie Curie	1867-11-07	1934-07-04	66	Chemist
7	Johann Gauss	1777-04-30	1855-02-23	77	Mathematician
2	Florence Nightingale	1820-05-12	1910-08-13	90	Nurse

- 시리즈 다루기 - 응용

- 시리즈를 데이터프레임으로 변환
- > `to_frame()` 메소드 사용

```
ages.to_frame()
```

Out :

Age	
0	37
1	61
2	90
3	66
4	56
5	45
6	41
7	77

• 시리즈 다루기 - 응용

- 브로드캐스팅 (Broadcasting)

> 시리즈와 같이 여러 개의 값을 가진 데이터(벡터)와 단순 크기를 나타내는 데이터(스칼라) 간의 연산을 지원하는 것

```
ages + ages
```

```
Out : 0    74  
      1   122  
      2   180  
      3   132  
      4   112  
      5    90  
      6    82  
      7   154
```

```
Name: Age, dtype: int64
```

자료형의 크기가 다른 두 자료의 연산을 허용하게 하는 것이
브로드캐스팅이다.

```
ages + 100
```

```
Out : 0   137  
      1   161  
      2   190  
      3   166  
      4   156  
      5   145  
      6   141  
      7   177
```

```
Name: Age, dtype: int64
```

• 시리즈 다루기 - 응용

- > 길이가 서로 다른 벡터를 연산하는 경우
 - 같은 인덱스의 값만 계산

```
ages + pd.Series([1, 100])
```

```
Out : 0      38.0  
      1     161.0  
      2      NaN  
      3      NaN  
      4      NaN  
      5      NaN  
      6      NaN  
      7      NaN  
      dtype: float64
```

0, 1 인덱스만 계산되고 나머지는 누락값(NaN)으로 처리

- 시리즈 다루기 - 응용

> `sort_index([ascending = False])`

```
rev_ages = ages.sort_index(ascending=False)
rev_ages
```

Out :

7	77
6	41
5	45
4	56
3	66
2	90
1	61
0	37

Name: Age, dtype: int64

• 시리즈 다루기 - 응용

> 정렬 후 연산

```
ages * 2
```

```
Out: 0    74  
     1   122  
     2   180  
     3   132  
     4   112  
     5    90  
     6    82  
     7   154  
     Name: Age, dtype: int64
```

=

```
ages + rev_ages
```

```
Out: 0    74  
     1   122  
     2   180  
     3   132  
     4   112  
     5    90  
     6    82  
     7   154  
     Name: Age, dtype: int64
```


- 데이터프레임 다루기

- > 브로드캐스팅 (Broadcasting)

- 시리즈와 같이 모든 요소를 대상으로 연산
 - 2를 곱하면 숫자는 2를 곱하고 문자열은 2배로 늘어남

```
scientists * 2
```

Out :

	Name		Born		Died		Age	Occupation	
0	Rosaline Franklin	Rosaline Franklin	1920-07-25	1920-07-25	1958-04-16	1958-04-16	74	Chemist	Chemist
1	William Gosset	William Gosset	1876-06-13	1876-06-13	1937-10-16	1937-10-16	122	Statistician	Statistician
2	Florence Nightingale	Florence Nightingale	1820-05-12	1820-05-12	1910-08-13	1910-08-13	180	Nurse	Nurse
3	Marie Curie	Marie Curie	1867-11-07	1867-11-07	1934-07-04	1934-07-04	132	Chemist	Chemist
4	Rachel Carson	Rachel Carson	1907-05-27	1907-05-27	1964-04-14	1964-04-14	112	Biologist	Biologist
5	John Snow	John Snow	1813-03-15	1813-03-15	1858-06-16	1858-06-16	90	Physician	Physician
6	Alan Turing	Alan Turing	1912-06-23	1912-06-23	1954-06-07	1954-06-07	82	Computer Scientist	Computer Scientist
7	Johann Gauss	Johann Gauss	1777-04-30	1777-04-30	1855-02-23	1855-02-23	154	Mathematician	Mathematician

- 데이터프레임 다루기

- > Boolean Array

- 데이터프레임의 Age 열의 평균보다 높은 데이터 출력

```
scientists = pd.read_csv('data/scientists.csv')  
scientists[scientists['Age'] > scientists['Age'].mean()]
```

Out :

	Name	Born	Died	Age	Occupation
1	William Gosset	1876-06-13	1937-10-16	61	Statistician
2	Florence Nightingale	1820-05-12	1910-08-13	90	Nurse
3	Marie Curie	1867-11-07	1934-07-04	66	Chemist
7	Johann Gauss	1777-04-30	1855-02-23	77	Mathematician

- 시리즈와 데이터프레임의 데이터 처리하기

- > 열의 자료형 바꾸기

- scientists 데이터프레임의 Age 열의 자료형 변환

```
print(scientists['Age'].dtype)
```

Out : int64

- Age 열의 int를 float 자료형으로 변경

```
scientists['Age'].astype(float)
```

Out :

0	37.0
1	61.0
2	90.0
3	66.0
4	56.0
5	45.0
6	41.0
7	77.0

Name: Age, dtype: float64

• 시리즈와 데이터프레임의 데이터 처리하기

> 열의 자료형 바꾸기

- scientists 데이터프레임의 Born과 Died 열의 자료형 확인

```
print(scientists['Born'].dtype)
print(scientists['Died'].dtype)
```

Out : object

object

- Born 열의 날짜 형식 문자열을 datetime 자료형으로 변경

```
born_datetime = pd.to_datetime(scientists['Born'], format='%Y-%m-%d')
print(born_datetime)
```

Out :

0	1920-07-25
1	1876-06-13
2	1820-05-12
3	1867-11-07
4	1907-05-27
5	1813-03-15
6	1912-06-23
7	1777-04-30

Name: Born, dtype: datetime64[ns]

• 시리즈와 데이터프레임의 데이터 처리하기

> 열의 자료형 바꾸기

- Died 열의 날짜 형식 문자열을 datetime 자료형으로 변경

```
died_datetime = pd.to_datetime(scientists['Died'], format='%Y-%m-%d')  
print(died_datetime)
```

```
Out : 0 1958-04-16  
      1 1937-10-16  
      2 1910-08-13  
      3 1934-07-04  
      4 1964-04-14  
      5 1858-06-16  
      6 1954-06-07  
      7 1855-02-23  
      Name: Died, dtype: datetime64[ns]
```

- astype으로 변경

```
print(scientists['Died'].astype('datetime64'))
```

```
Out : 0 1958-04-16  
      1 1937-10-16  
      2 1910-08-13  
      3 1934-07-04  
      4 1964-04-14  
      5 1858-06-16  
      6 1954-06-07  
      7 1855-02-23  
      Name: Died, dtype: datetime64[ns]
```

- 시리즈와 데이터프레임의 데이터 처리하기

- > 열의 자료형 바꾸기

- datetime으로 바뀐 2개의 자료를 새로운 열로 추가

```
scientists['born_dt'], scientists['died_dt']=(born_datetime, died_datetime)
scientists.head()
```

Out :

	Name	Born	Died	Age	Occupation	born_dt	died_dt
0	Rosaline Franklin	1920-07-25	1958-04-16	37	Chemist	1920-07-25	1958-04-16
1	William Gosset	1876-06-13	1937-10-16	61	Statistician	1876-06-13	1937-10-16
2	Florence Nightingale	1820-05-12	1910-08-13	90	Nurse	1820-05-12	1910-08-13
3	Marie Curie	1867-11-07	1934-07-04	66	Chemist	1867-11-07	1934-07-04
4	Rachel Carson	1907-05-27	1964-04-14	56	Biologist	1907-05-27	1964-04-14

- 시리즈와 데이터프레임의 데이터 처리하기

> datetime 시간 형식 지정자

지정자	설명	결과	지정자	설명	결과
%Y	년(4자리)	2002	%y	년(2자리)	02
%m	월	01-12	%B, %b	월(영어)	January, Jan
%d	일	01-31			
%H	시(24시간)	00-23	%I	시(12시간)	01-12
%M	분	00-59			
%S	초	00-59	%u	요일	1-7(월-일)
%w	요일	0-6(일-토)	%A, %a	요일(영어)	Sunday, Sun
%p	오전, 오후	AM, PM	%f	마이크로초	000000-999999
%z	UTC 차이	UTC+0900	%Z	기준 지역명	UTC, EST, ...
%j	올해 지난 일	001-366	%U	올해 지난 주	00-53
%c, %x	날짜와 시간				

• 시리즈와 데이터프레임의 데이터 처리하기

> 열의 자료형 바꾸기

- 과학자 삶의 시간 계산하기 (died_dt - born_dt)

```
scientists['age_days_dt']=scientists['died_dt']-scientists['born_dt']  
scientists['age_days_dt']
```

```
Out : 0    13779 days  
      1    22404 days  
      2    32964 days  
      3    24345 days  
      4    20777 days  
      5    16529 days  
      6    15324 days  
      7    28422 days  
      Name: age_days_dt, dtype: timedelta64[ns]
```


• 시리즈와 데이터프레임의 데이터 처리하기

> 열의 자료형 바꾸기

- datetime에서 연도, 월, 일, 분기 데이터 추출하기(dt 접근자)

```
print(scientists['born_dt'].dt.year)
print(scientists['born_dt'].dt.month)
print(scientists['born_dt'].dt.day)
print(scientists['born_dt'].dt.quarter)
```

Out :

0	1920	0	25
1	1876	1	13
2	1820	2	12
3	1867	3	7
4	1907	4	27
5	1813	5	15
6	1912	6	23
7	1777	7	30
Name: born_dt, dtype: int64		Name: born_dt, dtype: int64	
0	7	0	3
1	6	1	2
2	5	2	2
3	11	3	4
4	5	4	2
5	3	5	1
6	6	6	2
7	4	7	2
Name: born_dt, dtype: int64		Name: born_dt, dtype: int64	

• 시리즈와 데이터프레임의 데이터 처리하기

> 열의 자료형 바꾸기

- to_numeric 메소드로 error 제어

```
scientists.loc[[1, 3, 5, 7], 'Age'] = 'missing'
print(scientists['Age'].dtype)
scientists
```

Out : object

	Name	Born	Died	Age	Occupation	born_dt	died_dt	age_days_dt
0	Rosaline Franklin	1920-07-25	1958-04-16	37	Chemist	1920-07-25	1958-04-16	13779 days
1	William Gosset	1876-06-13	1937-10-16	missing	Statistician	1876-06-13	1937-10-16	22404 days
2	Florence Nightingale	1820-05-12	1910-08-13	90	Nurse	1820-05-12	1910-08-13	32964 days
3	Marie Curie	1867-11-07	1934-07-04	missing	Chemist	1867-11-07	1934-07-04	24345 days
4	Rachel Carson	1907-05-27	1964-04-14	56	Biologist	1907-05-27	1964-04-14	20777 days
5	John Snow	1813-03-15	1858-06-16	missing	Physician	1813-03-15	1858-06-16	16529 days
6	Alan Turing	1912-06-23	1954-06-07	41	Computer Scientist	1912-06-23	1954-06-07	15324 days
7	Johann Gauss	1777-04-30	1855-02-23	missing	Mathematician	1777-04-30	1855-02-23	28422 days

- 시리즈와 데이터프레임의 데이터 처리하기

- > 열의 자료형 바꾸기

- to_numeric 메소드로 error 제어

```
scientists['Age'].astype(int)
```

Out : `ValueError: invalid literal for int() with base 10: 'missing'`

옵션	설명
raise	숫자로 변환할 수 없는 값이 있으면 오류 발생 (기본값)
coerce	숫자로 변환할 수 없는 값을 누락값으로 지정
ignore	아무 작업도 하지 않음

• 시리즈와 데이터프레임의 데이터 처리하기

> 열의 자료형 바꾸기

- to_numeric 메소드로 error 제어

```
pd.to_numeric(scientists['Age'], errors = 'coerce')
```

```
Out : 0    37.0  
      1     NaN  
      2    90.0  
      3     NaN  
      4    56.0  
      5     NaN  
      6    41.0  
      7     NaN  
      Name: Age, dtype: float64
```

```
pd.to_numeric(scientists['Age'], errors = 'ignore')
```

```
Out : 0      37  
      1  missing  
      2     90  
      3  missing  
      4     56  
      5  missing  
      6     41  
      7  missing  
      Name: Age, dtype: object
```

• 시리즈와 데이터프레임의 데이터 처리하기

061

> 데이터프레임의 열 삭제하기

- `drop('열 이름', axis=1)`

```
scientists.drop('Age', axis=1).head(2)
```

Out :

	Name	Born	Died	Occupation	born_dt	died_dt	age_days_dt
0	Rosaline Franklin	1920-07-25	1958-04-16	Chemist	1920-07-25	1958-04-16	13779 days
1	William Gosset	1876-06-13	1937-10-16	Statistician	1876-06-13	1937-10-16	22404 days

- `drop('인덱스', axis=0)`

```
scientists.drop(0, axis=0).head(2)
```

Out :

	Name	Born	Died	Age	Occupation	born_dt	died_dt	age_days_dt
1	William Gosset	1876-06-13	1937-10-16	missing	Statistician	1876-06-13	1937-10-16	22404 days
2	Florence Nightingale	1820-05-12	1910-08-13	90	Nurse	1820-05-12	1910-08-13	32964 days

- 시리즈와 데이터프레임의 데이터 처리하기

- > 데이터프레임의 열 삭제하기

- 여러 개 한번에 삭제하기

```
scientists.drop(['Age','Occupation'],axis=1).head(2)
```

Out :

	Name	Born	Died	born_dt	died_dt	age_days_dt
0	Rosaline Franklin	1920-07-25	1958-04-16	1920-07-25	1958-04-16	13779 days
1	William Gosset	1876-06-13	1937-10-16	1876-06-13	1937-10-16	22404 days

• 데이터 저장하고 불러오기

> CSV, TSV로 저장하기

```
scientists.to_csv('data/scientists_df.csv')
```

Out :

```

1  ,Name,Born,Died,Age,Occupation,born_dt,died_dt,age_days_dt
2  0,Rosaline Franklin,1920-07-25,1958-04-16,66,Chemist,1920-07-25,1958-04-16,13779 days
3  1,William Gosset,1876-06-13,1937-10-16,56,Statistician,1876-06-13,1937-10-16,22404 days
4  2,Florence Nightingale,1820-05-12,1910-08-13,41,Nurse,1820-05-12,1910-08-13,32964 days
5  3,Marie Curie,1867-11-07,1934-07-04,77,Chemist,1867-11-07,1934-07-04,24345 days
6  4,Rachel Carson,1907-05-27,1964-04-14,90,Biologist,1907-05-27,1964-04-14,20777 days
7  5,John Snow,1813-03-15,1858-06-16,45,Physician,1813-03-15,1858-06-16,16529 days
8  6,Alan Turing,1912-06-23,1954-06-07,37,Computer Scientist,1912-06-23,1954-06-07,15324 days
9  7,Johann Gauss,1777-04-30,1855-02-23,61,Mathematician,1777-04-30,1855-02-23,28422 days
```

```
scientists.to_csv('data/scientists_df.tsv', sep='\t')
```

Out :

```

1  \tName\tBorn\tDied\tAge\tOccupation\tborn_dt\tdied_dt\tage_days_dt
2  0\tRosaline Franklin\t1920-07-25\t1958-04-16\t66\tChemist\t1920-07-25\t1958-04-16\t13779 days
3  1\tWilliam Gosset\t1876-06-13\t1937-10-16\t56\tStatistician\t1876-06-13\t1937-10-16\t22404 days
4  2\tFlorence Nightingale\t1820-05-12\t1910-08-13\t41\tNurse\t1820-05-12\t1910-08-13\t32964 days
5  3\tMarie Curie\t1867-11-07\t1934-07-04\t77\tChemist\t1867-11-07\t1934-07-04\t24345 days
6  4\tRachel Carson\t1907-05-27\t1964-04-14\t90\tBiologist\t1907-05-27\t1964-04-14\t20777 days
7  5\tJohn Snow\t1813-03-15\t1858-06-16\t45\tPhysician\t1813-03-15\t1858-06-16\t16529 days
8  6\tAlan Turing\t1912-06-23\t1954-06-07\t37\tComputer Scientist\t1912-06-23\t1954-06-07\t15324 days
9  7\tJohann Gauss\t1777-04-30\t1855-02-23\t61\tMathematician\t1777-04-30\t1855-02-23\t28422 days
```

- 데이터 저장하고 불러오기

> CSV, TSV 데이터프레임으로 불러오기

```
pd.read_csv('data/scientists_df.csv')
```

Out :

	Unnamed: 0	Name	Born	Died	Age	Occupation	born_dt	died_dt	age_days_dt
0	0	Rosaline Franklin	1920-07-25	1958-04-16	66	Chemist	1920-07-25	1958-04-16	13779 days
1	1	William Gosset	1876-06-13	1937-10-16	56	Statistician	1876-06-13	1937-10-16	22404 days
2	2	Florence Nightingale	1820-05-12	1910-08-13	41	Nurse	1820-05-12	1910-08-13	32964 days
3	3	Marie Curie	1867-11-07	1934-07-04	77	Chemist	1867-11-07	1934-07-04	24345 days
4	4	Rachel Carson	1907-05-27	1964-04-14	90	Biologist	1907-05-27	1964-04-14	20777 days
5	5	John Snow	1813-03-15	1858-06-16	45	Physician	1813-03-15	1858-06-16	16529 days
6	6	Alan Turing	1912-06-23	1954-06-07	37	Computer Scientist	1912-06-23	1954-06-07	15324 days
7	7	Johann Gauss	1777-04-30	1855-02-23	61	Mathematician	1777-04-30	1855-02-23	28422 days

```
pd.read_csv('data/scientists_df.tsv', sep='\\t')
```

Out :

	Unnamed: 0	Name	Born	Died	Age	Occupation	born_dt	died_dt	age_days_dt
0	0	Rosaline Franklin	1920-07-25	1958-04-16	66	Chemist	1920-07-25	1958-04-16	13779 days
1	1	William Gosset	1876-06-13	1937-10-16	56	Statistician	1876-06-13	1937-10-16	22404 days
2	2	Florence Nightingale	1820-05-12	1910-08-13	41	Nurse	1820-05-12	1910-08-13	32964 days
3	3	Marie Curie	1867-11-07	1934-07-04	77	Chemist	1867-11-07	1934-07-04	24345 days
4	4	Rachel Carson	1907-05-27	1964-04-14	90	Biologist	1907-05-27	1964-04-14	20777 days
5	5	John Snow	1813-03-15	1858-06-16	45	Physician	1813-03-15	1858-06-16	16529 days
6	6	Alan Turing	1912-06-23	1954-06-07	37	Computer Scientist	1912-06-23	1954-06-07	15324 days
7	7	Johann Gauss	1777-04-30	1855-02-23	61	Mathematician	1777-04-30	1855-02-23	28422 days

- 데이터 저장하고 불러오기

> 특정 컬럼을 인덱스로 지정하여 불러오기

```
pd.read_csv('data/scientists_df.csv', index_col = 0)
```

Out :

	Unnamed: 0	Born	Died	Age	Occupation	born_dt	died_dt	age_days_dt
Name								
Rosaline Franklin	0	1920-07-25	1958-04-16	66	Chemist	1920-07-25	1958-04-16	13779 days
William Gosset	1	1876-06-13	1937-10-16	56	Statistician	1876-06-13	1937-10-16	22404 days
Florence Nightingale	2	1820-05-12	1910-08-13	41	Nurse	1820-05-12	1910-08-13	32964 days
Marie Curie	3	1867-11-07	1934-07-04	77	Chemist	1867-11-07	1934-07-04	24345 days
Rachel Carson	4	1907-05-27	1964-04-14	90	Biologist	1907-05-27	1964-04-14	20777 days
John Snow	5	1813-03-15	1858-06-16	45	Physician	1813-03-15	1858-06-16	16529 days
Alan Turing	6	1912-06-23	1954-06-07	37	Computer Scientist	1912-06-23	1954-06-07	15324 days
Johann Gauss	7	1777-04-30	1855-02-23	61	Mathematician	1777-04-30	1855-02-23	28422 days

```
pd.read_csv('data/scientists_df.csv', index_col = 'Name')
```

Out :

	Unnamed: 0	Born	Died	Age	Occupation	born_dt	died_dt	age_days_dt
Name								
Rosaline Franklin	0	1920-07-25	1958-04-16	66	Chemist	1920-07-25	1958-04-16	13779 days
William Gosset	1	1876-06-13	1937-10-16	56	Statistician	1876-06-13	1937-10-16	22404 days
Florence Nightingale	2	1820-05-12	1910-08-13	41	Nurse	1820-05-12	1910-08-13	32964 days
Marie Curie	3	1867-11-07	1934-07-04	77	Chemist	1867-11-07	1934-07-04	24345 days
Rachel Carson	4	1907-05-27	1964-04-14	90	Biologist	1907-05-27	1964-04-14	20777 days
John Snow	5	1813-03-15	1858-06-16	45	Physician	1813-03-15	1858-06-16	16529 days
Alan Turing	6	1912-06-23	1954-06-07	37	Computer Scientist	1912-06-23	1954-06-07	15324 days
Johann Gauss	7	1777-04-30	1855-02-23	61	Mathematician	1777-04-30	1855-02-23	28422 days

• 데이터 저장하고 불러오기

> 인덱스를 제외하고 저장하기

```
scientists.to_csv('data/scientists_notindex.csv', index=False)
pd.read_csv('data/scientists_notindex.csv')
```

Out :

	Name	Born	Died	Age	Occupation	born_dt	died_dt	age_days_dt
0	Rosaline Franklin	1920-07-25	1958-04-16	37	Chemist	1920-07-25	1958-04-16	13779 days
1	William Gosset	1876-06-13	1937-10-16	61	Statistician	1876-06-13	1937-10-16	22404 days
2	Florence Nightingale	1820-05-12	1910-08-13	90	Nurse	1820-05-12	1910-08-13	32964 days
3	Marie Curie	1867-11-07	1934-07-04	66	Chemist	1867-11-07	1934-07-04	24345 days
4	Rachel Carson	1907-05-27	1964-04-14	56	Biologist	1907-05-27	1964-04-14	20777 days
5	John Snow	1813-03-15	1858-06-16	45	Physician	1813-03-15	1858-06-16	16529 days
6	Alan Turing	1912-06-23	1954-06-07	41	Computer Scientist	1912-06-23	1954-06-07	15324 days
7	Johann Gauss	1777-04-30	1855-02-23	77	Mathematician	1777-04-30	1855-02-23	28422 days

- 데이터 저장하고 불러오기

> datetime 데이터 불러오면서 지정하기

```
scientists = pd.read_csv('scientists.csv', parse_dates=['Born', 'Died'])
scientists.info()
```

Out : <class 'pandas.core.frame.DataFrame'>
RangeIndex: 8 entries, 0 to 7
Data columns (total 5 columns):
Column Non-Null Count Dtype
--- ---
0 Name 8 non-null object
1 Born 8 non-null datetime64[ns]
2 Died 8 non-null datetime64[ns]
3 Age 8 non-null int64
4 Occupation 8 non-null object
dtypes: datetime64[ns](2), int64(1), object(2)
memory usage: 448.0+ bytes

- 데이터베이스 불러오기

- > 데이터베이스 불러오기

```
import pymysql
import pandas as pd

conn = pymysql.connect(host="localhost", user="root",
                        password="1234", port=3306,
                        database="mydb")

users = pd.read_sql('select * from users', conn)
orders = pd.read_sql('select * from orders', conn)

conn.close()
```

- 데이터베이스 불러오기

> 데이터베이스 불러오기

```
users.head()
```

Out :

	id	name	email
0	1	김철수	chulsoo.kim@example.com
1	2	이영희	younghee.lee@example.com
2	3	박민수	minsoo@example.com
3	4	최지은	jieun.choi@example.com
4	5	한동훈	donghoon.han@example.com

```
orders.head()
```

Out :

	order_id	user_id	order_date	amount
0	1	1	2024-02-01 10:15:00	50000.00
1	2	2	2024-02-02 12:30:00	75000.50
2	3	3	2024-02-03 14:45:00	62000.75
3	4	4	2024-02-04 09:20:00	81000.20
4	5	5	2024-02-05 16:10:00	92000.30

• 데이터베이스 불러오기

> 데이터베이스(SQLAlchemy) 불러오기

```
import sqlalchemy as sql
import pandas as pd

engine = sql.create_engine('mysql+pymysql://root:1234@localhost:3306/mydb')

users = pd.read_sql('select * from users', engine)
orders = pd.read_sql('select * from orders', engine)
```

> **mysql+pymysql://root:1234@localhost:3306/mydb**

프로토콜://계정:비밀번호@도메인:포트/데이터베이스

• 데이터 연결 기초

csv 데이터가 연단위로 분할이 되어 있어서,
하나로 합쳐주는 과정이 필요하다.

- > Concat 메서드로 데이터 연결하기
 - 데이터 준비(csv로 파일 읽어오기)

```
import pandas as pd
df1 = pd.read_csv('data/concat_1.csv')
df2 = pd.read_csv('data/concat_2.csv')
df3 = pd.read_csv('data/concat_3.csv')
```

Out :

	df1				df2				df3					
	A	B	C	D	A	B	C	D	A	B	C	D		
0	a0	b0	c0	d0	0	a4	b4	c4	d4	0	a8	b8	c8	d8
1	a1	b1	c1	d1	1	a5	b5	c5	d5	1	a9	b9	c9	d9
2	a2	b2	c2	d2	2	a6	b6	c6	d6	2	a10	b10	c10	d10
3	a3	b3	c3	d3	3	a7	b7	c7	d7	3	a11	b11	c11	d11

• 데이터 연결 기초

> Concat 메서드로 데이터 연결하기

- 데이터 연결

```
row_concat = pd.concat([df1, df2, df3])  
row_concat
```

Out :

	A	B	C	D
0	a0	b0	c0	d0
1	a1	b1	c1	d1
2	a2	b2	c2	d2
3	a3	b3	c3	d3
0	a4	b4	c4	d4
1	a5	b5	c5	d5
2	a6	b6	c6	d6
3	a7	b7	c7	d7
0	a8	b8	c8	d8
1	a9	b9	c9	d9
2	a10	b10	c10	d10
3	a11	b11	c11	d11

• 데이터 연결 기초

> Concat 메서드로 데이터 연결하기

- iloc로 4번째 행 확인

```
row_concat.iloc[3]
```

Out :

A	a3
B	b3
C	c3
D	d3

Name: 3, dtype: object

- loc로 3 인덱스 확인

```
row_concat.loc[3]
```

Out :

	A	B	C	D
3	a3	b3	c3	d3
3	a7	b7	c7	d7
3	a11	b11	c11	d11

- 데이터 연결 기초

> Concat 메서드를 사용하여 열 방향으로 연결하기

- axis = 1

```
col_concat = pd.concat([df1, df2, df3], axis = 1)
col_concat
```

Out :

	A	B	C	D	A	B	C	D	A	B	C	D
0	a0	b0	c0	d0	a4	b4	c4	d4	a8	b8	c8	d8
1	a1	b1	c1	d1	a5	b5	c5	d5	a9	b9	c9	d9
2	a2	b2	c2	d2	a6	b6	c6	d6	a10	b10	c10	d10
3	a3	b3	c3	d3	a7	b7	c7	d7	a11	b11	c11	d11

```
col_concat['A']
```

Out :

	A	A	A
0	a0	a4	a8
1	a1	a5	a9
2	a2	a6	a10
3	a3	a7	a11

• 데이터 연결 기초

> ignore_index = True

- 데이터프레임을 연결할 때, 인덱스를 초기화시키는 옵션

```
pd.concat([df1, df2, df3], ignore_index = True)
```

Out :

	A	B	C	D
0	a0	b0	c0	d0
1	a1	b1	c1	d1
2	a2	b2	c2	d2
3	a3	b3	c3	d3
4	a4	b4	c4	d4
5	a5	b5	c5	d5
6	a6	b6	c6	d6
7	a7	b7	c7	d7
8	a8	b8	c8	d8
9	a9	b9	c9	d9
10	a10	b10	c10	d10
11	a11	b11	c11	d11

- 데이터 연결 기초

> Concat 메서드를 사용하여 열 방향으로 연결하기

- axis = 1

```
col_concat = pd.concat([df1, df2, df3], axis = 1, ignore_index = True)
col_concat
```

Out :

	0	1	2	3	4	5	6	7	8	9	10	11
0	a0	b0	c0	d0	a4	b4	c4	d4	a8	b8	c8	d8
1	a1	b1	c1	d1	a5	b5	c5	d5	a9	b9	c9	d9
2	a2	b2	c2	d2	a6	b6	c6	d6	a10	b10	c10	d10
3	a3	b3	c3	d3	a7	b7	c7	d7	a11	b11	c11	d11

• 데이터 연결 기초

> Concat 메서드를 사용하여 열 방향으로 연결하기

- `df.columns = []` 로 컬럼명 변경

```
col_concat.columns = ['A','B','C','D','E','F','G','H','I','J','K','L']  
col_concat
```

Out :

	A	B	C	D	E	F	G	H	I	J	K	L
0	a0	b0	c0	d0	a4	b4	c4	d4	a8	b8	c8	d8
1	a1	b1	c1	d1	a5	b5	c5	d5	a9	b9	c9	d9
2	a2	b2	c2	d2	a6	b6	c6	d6	a10	b10	c10	d10
3	a3	b3	c3	d3	a7	b7	c7	d7	a11	b11	c11	d11

- 데이터 연결 기초

- > 다양한 방법으로 데이터 연결하기
 - 공통 열과 인덱스로 연결하기

```
df1.columns = ['A', 'B', 'C', 'D']
df2.columns = ['E', 'F', 'G', 'H']
df3.columns = ['A', 'C', 'F', 'H']
```

Out :

	df1				df2				df3					
	A	B	C	D		E	F	G	H		A	C	F	H
0	a0	b0	c0	d0	0	a4	b4	c4	d4	0	a8	b8	c8	d8
1	a1	b1	c1	d1	1	a5	b5	c5	d5	1	a9	b9	c9	d9
2	a2	b2	c2	d2	2	a6	b6	c6	d6	2	a10	b10	c10	d10
3	a3	b3	c3	d3	3	a7	b7	c7	d7	3	a11	b11	c11	d11

• 데이터 연결 기초

> 다양한 방법으로 데이터 연결하기

- 공통 열과 인덱스로 연결하기

```
pd.concat([df1, df2, df3])
```

Out :

	A	B	C	D	E	F	G	H
0	a0	b0	c0	d0	NaN	NaN	NaN	NaN
1	a1	b1	c1	d1	NaN	NaN	NaN	NaN
2	a2	b2	c2	d2	NaN	NaN	NaN	NaN
3	a3	b3	c3	d3	NaN	NaN	NaN	NaN
0	NaN	NaN	NaN	NaN	a4	b4	c4	d4
1	NaN	NaN	NaN	NaN	a5	b5	c5	d5
2	NaN	NaN	NaN	NaN	a6	b6	c6	d6
3	NaN	NaN	NaN	NaN	a7	b7	c7	d7
0	a8	NaN	b8	NaN	NaN	c8	NaN	d8
1	a9	NaN	b9	NaN	NaN	c9	NaN	d9
2	a10	NaN	b10	NaN	NaN	c10	NaN	d10
3	a11	NaN	b11	NaN	NaN	c11	NaN	d11

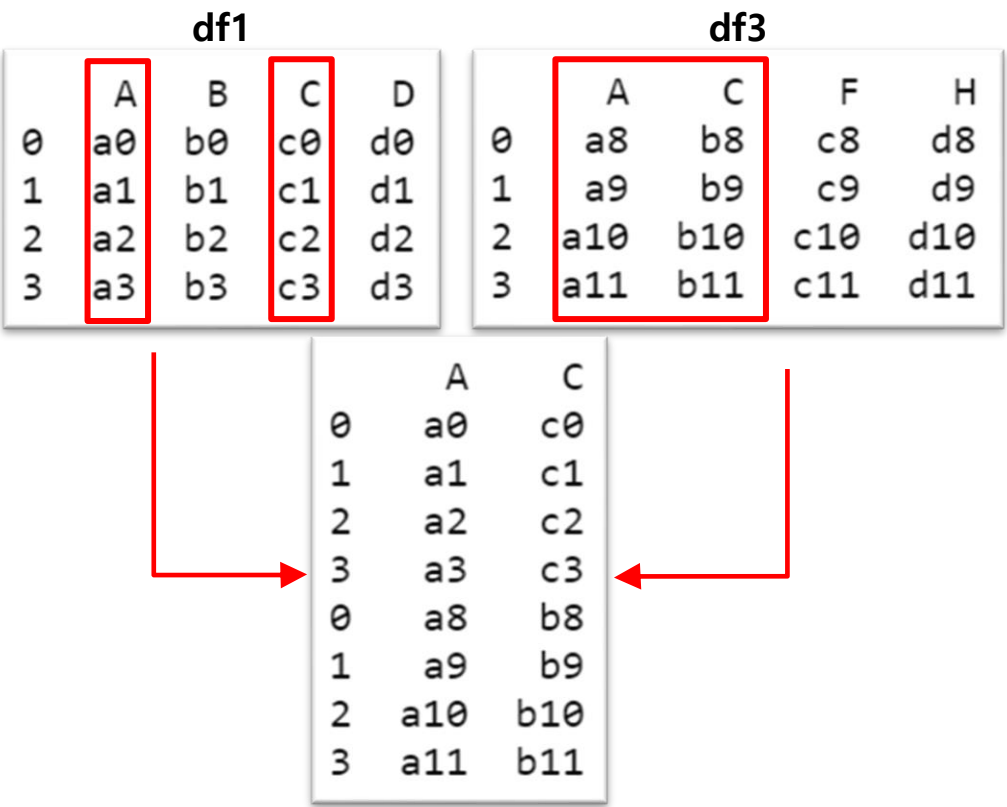
• 데이터 연결 기초

- > 다양한 방법으로 데이터 연결하기
 - 공통 열과 인덱스로 연결하기(join)

```
pd.concat([df1, df3], join='inner')
```

Out :

	A	C
0	a0	c0
1	a1	c1
2	a2	c2
3	a3	c3
0	a8	b8
1	a9	b9
2	a10	b10
3	a11	b11



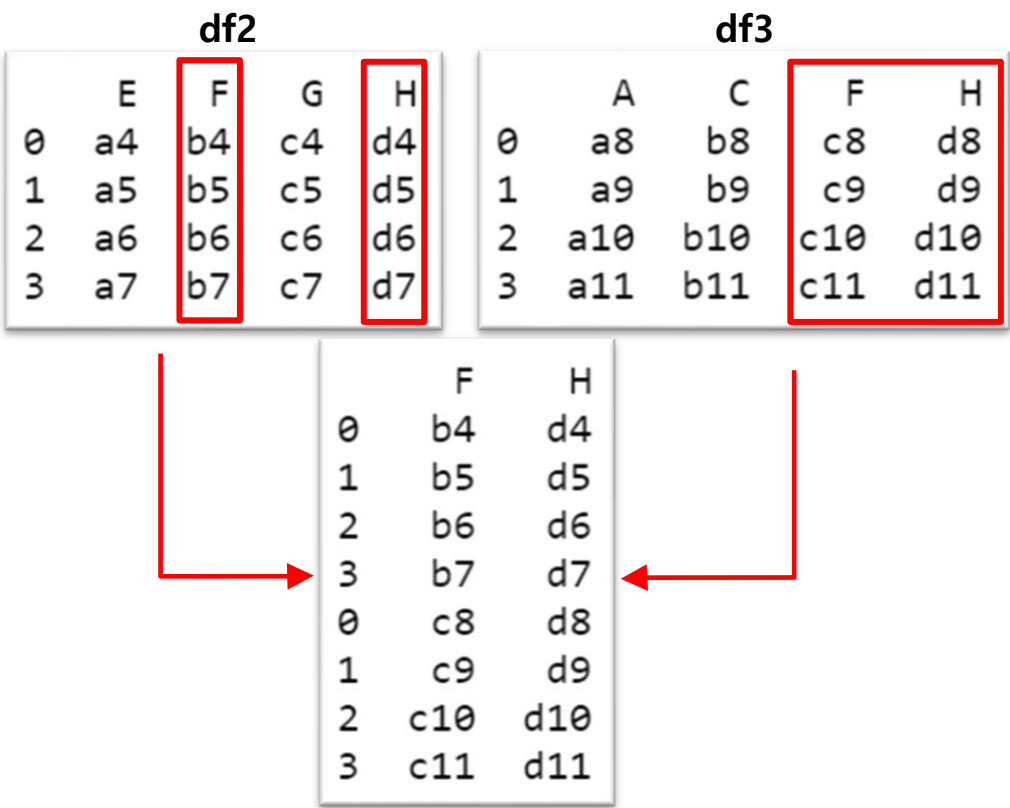
• 데이터 연결 기초

- > 다양한 방법으로 데이터 연결하기
 - 공통 열과 인덱스로 연결하기(join)

```
pd.concat([df2, df3], join='inner')
```

Out :

	F	H
0	b4	d4
1	b5	d5
2	b6	d6
3	b7	d7
0	c8	d8
1	c9	d9
2	c10	d10
3	c11	d11



• 데이터 연결 기초

- > merge 메서드를 사용한 데이터 연결하기
 - 서로 같은 키를 가지고 있는 경우

```
person = pd.read_csv('data/survey_person.csv')
site = pd.read_csv('data/survey_site.csv')
visited = pd.read_csv('data/survey_visited.csv')
survey = pd.read_csv('data/survey_survey.csv')
```

• 데이터 연결 기초

- > merge 메서드를 사용한 데이터 연결하기
 - 서로 같은 키를 가지고 있는 경우

person(관측자)

	ident	personal	family
0	dye	William	Dyer
1	pb	Frank	Pabodie
2	lake	Anderson	Lake
3	roe	Valentina	Roerich
4	danforth	Frank	Danforth

site(장소)

	name	lat	long
0	DR-1	-49.85	-128.57
1	DR-3	-47.15	-126.72
2	MSK-4	-48.87	-123.40

visited(날짜)

	ident	site	dated
0	619	DR-1	1927-02-08
1	622	DR-1	1927-02-10
2	734	DR-3	1939-01-07
3	735	DR-3	1930-01-12
4	751	DR-3	1930-02-26
5	752	DR-3	NaN
6	837	MSK-4	1932-01-14
7	844	DR-1	1932-03-22

survey(날씨정보)

	taken	person	quant	reading
0	619	dye	rad	9.82
1	619	dye	sal	0.13
2	622	dye	rad	7.80
3	622	dye	sal	0.09
4	734	pb	rad	8.41
5	734	lake	sal	0.05
6	734	pb	temp	-21.50
7	735	pb	rad	7.22
8	735	NaN	sal	0.06
9	735	NaN	temp	-26.00
10	751	pb	rad	4.35
11	751	pb	temp	-18.50
12	751	lake	sal	0.10
13	752	lake	rad	2.19
14	752	lake	sal	0.09
15	752	lake	temp	-16.00
16	752	roe	sal	41.60
17	837	lake	rad	1.46
18	837	lake	sal	0.21
19	837	roe	sal	22.50
20	844	roe	rad	11.25

• 데이터 연결 기초

> merge 메서드를 사용한 데이터 연결하기

- left_on, right_on을 사용하여 공통 키 선택

```
sv_merge = site.merge(visited, left_on='name', right_on='site')
sv_merge
```

Out :

	name	lat	long	ident	site	dated
0	DR-1	-49.85	-128.57	619	DR-1	1927-02-08
1	DR-1	-49.85	-128.57	622	DR-1	1927-02-10
2	DR-1	-49.85	-128.57	844	DR-1	1932-03-22
3	DR-3	-47.15	-126.72	734	DR-3	1939-01-07
4	DR-3	-47.15	-126.72	735	DR-3	1930-01-12
5	DR-3	-47.15	-126.72	751	DR-3	1930-02-26
6	DR-3	-47.15	-126.72	752	DR-3	NaN
7	MSK-4	-48.87	-123.40	837	MSK-4	1932-01-14

• 데이터 연결 기초

> merge 메서드를 사용한 데이터 연결하기

- left_on, right_on을 사용하여 공통 키 선택

```
sv_merge = site.merge(visited, left_on='name', right_on='site')
sv_merge
```

Out :

	name	lat	long		ident	site	dated
0	DR-1	-49.85	-128.57	0	619	DR-1	1927-02-08
1	DR-3	-47.15	-126.72	1	622	DR-1	1927-02-10
2	MSK-4	-48.87	-123.40	2	734	DR-3	1939-01-07
				3	735	DR-3	1930-01-12
				4	751	DR-3	1930-02-26
				5	752	DR-3	NaN
				6	837	MSK-4	1932-01-14
				7	844	DR-1	1932-03-22

- 데이터 연결 기초

- > merge 메서드를 사용한 데이터 연결하기
 - left_on, right_on을 사용하여 공통 키 선택

```
ps_merge = person.merge(survey, left_on='ident', right_on='person')
ps_merge
```

Out :

	ident	personal	family	taken	person	quant	reading
0	dyer	William	Dyer	619	dyer	rad	9.82
1	dyer	William	Dyer	619	dyer	sal	0.13
2	dyer	William	Dyer	622	dyer	rad	7.80
3	dyer	William	Dyer	622	dyer	sal	0.09
4	pb	Frank	Pabodie	734	pb	rad	8.41
5	pb	Frank	Pabodie	734	pb	temp	-21.50
6	pb	Frank	Pabodie	735	pb	rad	7.22
7	pb	Frank	Pabodie	751	pb	rad	4.35
8	pb	Frank	Pabodie	751	pb	temp	-18.50
9	lake	Anderson	Lake	734	lake	sal	0.05
10	lake	Anderson	Lake	751	lake	sal	0.10
11	lake	Anderson	Lake	752	lake	rad	2.19
12	lake	Anderson	Lake	752	lake	sal	0.09
13	lake	Anderson	Lake	752	lake	temp	-16.00
14	lake	Anderson	Lake	837	lake	rad	1.46
15	lake	Anderson	Lake	837	lake	sal	0.21
16	roe	Valentina	Roerich	752	roe	sal	41.60
17	roe	Valentina	Roerich	837	roe	sal	22.50
18	roe	Valentina	Roerich	844	roe	rad	11.25

• 데이터 연결 기초

- > merge 메서드를 사용한 데이터 연결하기
 - left_on, right_on을 사용하여 공통 키 선택

```
vs_merge = visited.merge(survey, left_on='ident', right_on='taken')
vs_merge
```

Out :

	ident	site	dated	taken	person	quant	reading
0	619	DR-1	1927-02-08	619	dye	rad	9.82
1	619	DR-1	1927-02-08	619	dye	sal	0.13
2	622	DR-1	1927-02-10	622	dye	rad	7.80
3	622	DR-1	1927-02-10	622	dye	sal	0.09
4	734	DR-3	1939-01-07	734	pb	rad	8.41
5	734	DR-3	1939-01-07	734	lake	sal	0.05
6	734	DR-3	1939-01-07	734	pb	temp	-21.50
7	735	DR-3	1930-01-12	735	pb	rad	7.22
8	735	DR-3	1930-01-12	735	NaN	sal	0.06
9	735	DR-3	1930-01-12	735	NaN	temp	-26.00
10	751	DR-3	1930-02-26	751	pb	rad	4.35
11	751	DR-3	1930-02-26	751	pb	temp	-18.50
12	751	DR-3	1930-02-26	751	lake	sal	0.10
13	752	DR-3	NaN	752	lake	rad	2.19
14	752	DR-3	NaN	752	lake	sal	0.09
15	752	DR-3	NaN	752	lake	temp	-16.00
16	752	DR-3	NaN	752	roe	sal	41.60
17	837	MSK-4	1932-01-14	837	lake	rad	1.46
18	837	MSK-4	1932-01-14	837	lake	sal	0.21
19	837	MSK-4	1932-01-14	837	roe	sal	22.50
20	844	DR-1	1932-03-22	844	roe	rad	11.25

• 데이터 연결 기초

- > merge 메서드를 사용한 데이터 연결하기
 - 서로 같은 키를 가지고 있는 경우

person(관측자)

	ident	personal	family
0	dyer	William	Dyer
1	pb	Frank	Pabodie
2	lake	Anderson	Lake
3	roe	Valentina	Roerich
4	danforth	Frank	Danforth

site(장소)

	name	lat	long
0	DR-1	-49.85	-128.57
1	DR-3	-47.15	-126.72
2	MSK-4	-48.87	-123.40

visited(날짜)

	ident	site	dated
0	619	DR-1	1927-02-08
1	622	DR-1	1927-02-10
2	734	DR-3	1939-01-07
3	735	DR-3	1930-01-12
4	751	DR-3	1930-02-26
5	752	DR-3	NaN
6	837	MSK-4	1932-01-14
7	844	DR-1	1932-03-22

survey(날씨정보)

	taken	person	quant	reading
0	619	dye	rad	9.82
1	619	dye	sal	0.13
2	622	dye	rad	7.80
3	622	dye	sal	0.09
4	734	pb	rad	8.41
5	734	lake	sal	0.05
6	734	pb	temp	-21.50
7	735	pb	rad	7.22
8	735	NaN	sal	0.06
9	735	NaN	temp	-26.00
10	751	pb	rad	4.35
11	751	pb	temp	-18.50
12	751	lake	sal	0.10
13	752	lake	rad	2.19
14	752	lake	sal	0.09
15	752	lake	temp	-16.00
16	752	roe	sal	41.60
17	837	lake	rad	1.46
18	837	lake	sal	0.21
19	837	roe	sal	22.50
20	844	roe	rad	11.25

• 데이터 연결 기초

> merge 메서드를 사용한 데이터 연결하기

- 4개의 데이터 모두 연결하기(survey + person)

```
sp = survey.merge(person, left_on='person', right_on='ident')
sp
```

Out :

	taken	person	quant	reading	ident	personal	family
0	619	dye	rad	9.82	dye	William	Dye
1	619	dye	sal	0.13	dye	William	Dye
2	622	dye	rad	7.80	dye	William	Dye
3	622	dye	sal	0.09	dye	William	Dye
4	734	pb	rad	8.41	pb	Frank	Pabodie
5	734	pb	temp	-21.50	pb	Frank	Pabodie
6	735	pb	rad	7.22	pb	Frank	Pabodie
7	751	pb	rad	4.35	pb	Frank	Pabodie
8	751	pb	temp	-18.50	pb	Frank	Pabodie
9	734	lake	sal	0.05	lake	Anderson	Lake
10	751	lake	sal	0.10	lake	Anderson	Lake
11	752	lake	rad	2.19	lake	Anderson	Lake
12	752	lake	sal	0.09	lake	Anderson	Lake
13	752	lake	temp	-16.00	lake	Anderson	Lake
14	837	lake	rad	1.46	lake	Anderson	Lake
15	837	lake	sal	0.21	lake	Anderson	Lake
16	752	roe	sal	41.60	roe	Valentina	Roerich
17	837	roe	sal	22.50	roe	Valentina	Roerich
18	844	roe	rad	11.25	roe	Valentina	Roerich

• 데이터 연결 기초

> merge 메서드를 사용한 데이터 연결하기

- 4개의 데이터 모두 연결하기(survey + person + visited)

```
spv = sp.merge(visited, left_on = 'taken', right_on='ident')
spv
```

Out :

	taken	person	quant	reading	ident_x	personal	family	ident_y	site	dated
0	619	dyer	rad	9.82	dyer	William	Dyer	619	DR-1	1927-02-08
1	619	dyer	sal	0.13	dyer	William	Dyer	619	DR-1	1927-02-08
2	622	dyer	rad	7.80	dyer	William	Dyer	622	DR-1	1927-02-10
3	622	dyer	sal	0.09	dyer	William	Dyer	622	DR-1	1927-02-10
4	734	pb	rad	8.41	pb	Frank	Pabodie	734	DR-3	1939-01-07
5	734	pb	temp	-21.50	pb	Frank	Pabodie	734	DR-3	1939-01-07
6	734	lake	sal	0.05	lake	Anderson	Lake	734	DR-3	1939-01-07
7	735	pb	rad	7.22	pb	Frank	Pabodie	735	DR-3	1930-01-12
8	751	pb	rad	4.35	pb	Frank	Pabodie	751	DR-3	1930-02-26
9	751	pb	temp	-18.50	pb	Frank	Pabodie	751	DR-3	1930-02-26
10	751	lake	sal	0.10	lake	Anderson	Lake	751	DR-3	1930-02-26
11	752	lake	rad	2.19	lake	Anderson	Lake	752	DR-3	NaN
12	752	lake	sal	0.09	lake	Anderson	Lake	752	DR-3	NaN
13	752	lake	temp	-16.00	lake	Anderson	Lake	752	DR-3	NaN
14	752	roe	sal	41.60	roe	Valentina	Roerich	752	DR-3	NaN
15	837	lake	rad	1.46	lake	Anderson	Lake	837	MSK-4	1932-01-14
16	837	lake	sal	0.21	lake	Anderson	Lake	837	MSK-4	1932-01-14
17	837	roe	sal	22.50	roe	Valentina	Roerich	837	MSK-4	1932-01-14
18	844	roe	rad	11.25	roe	Valentina	Roerich	844	DR-1	1932-03-22

• 데이터 연결 기초

- > merge 메서드를 사용한 데이터 연결하기
 - 4개의 데이터 모두 연결하기(survey + person + visited + site)

```
spvs = spv.merge(site, left_on = 'site', right_on='name')
spvs
```

Out :

	taken	person	quant	reading	ident_x	personal	family	ident_y	site	dated	name	lat	long
0	619	dyer	rad	9.82	dyer	William	Dyer	619	DR-1	1927-02-08	DR-1	-49.85	-128.57
1	619	dyer	sal	0.13	dyer	William	Dyer	619	DR-1	1927-02-08	DR-1	-49.85	-128.57
2	622	dyer	rad	7.80	dyer	William	Dyer	622	DR-1	1927-02-10	DR-1	-49.85	-128.57
3	622	dyer	sal	0.09	dyer	William	Dyer	622	DR-1	1927-02-10	DR-1	-49.85	-128.57
4	844	roe	rad	11.25	roe	Valentina	Roerich	844	DR-1	1932-03-22	DR-1	-49.85	-128.57
5	734	pb	rad	8.41	pb	Frank	Pabodie	734	DR-3	1939-01-07	DR-3	-47.15	-126.72
6	734	pb	temp	-21.50	pb	Frank	Pabodie	734	DR-3	1939-01-07	DR-3	-47.15	-126.72
7	734	lake	sal	0.05	lake	Anderson	Lake	734	DR-3	1939-01-07	DR-3	-47.15	-126.72
8	735	pb	rad	7.22	pb	Frank	Pabodie	735	DR-3	1930-01-12	DR-3	-47.15	-126.72
9	751	pb	rad	4.35	pb	Frank	Pabodie	751	DR-3	1930-02-26	DR-3	-47.15	-126.72
10	751	pb	temp	-18.50	pb	Frank	Pabodie	751	DR-3	1930-02-26	DR-3	-47.15	-126.72
11	751	lake	sal	0.10	lake	Anderson	Lake	751	DR-3	1930-02-26	DR-3	-47.15	-126.72
12	752	lake	rad	2.19	lake	Anderson	Lake	752	DR-3	NaN	DR-3	-47.15	-126.72
13	752	lake	sal	0.09	lake	Anderson	Lake	752	DR-3	NaN	DR-3	-47.15	-126.72
14	752	lake	temp	-16.00	lake	Anderson	Lake	752	DR-3	NaN	DR-3	-47.15	-126.72
15	752	roe	sal	41.60	roe	Valentina	Roerich	752	DR-3	NaN	DR-3	-47.15	-126.72
16	837	lake	rad	1.46	lake	Anderson	Lake	837	MSK-4	1932-01-14	MSK-4	-48.87	-123.40
17	837	lake	sal	0.21	lake	Anderson	Lake	837	MSK-4	1932-01-14	MSK-4	-48.87	-123.40
18	837	roe	sal	22.50	roe	Valentina	Roerich	837	MSK-4	1932-01-14	MSK-4	-48.87	-123.40

• 데이터 연결 기초

> merge 메서드를 사용한 데이터 연결하기

- 중복 컬럼 제거(ident_x, ident_y, name)

```
spvs.drop(['ident_x', 'ident_y', 'name'], axis = 1)
```

Out :

	taken	person	quant	reading	personal	family	site	dated	lat	long
0	619	dyer	rad	9.82	William	Dyer	DR-1	1927-02-08	-49.85	-128.57
1	619	dyer	sal	0.13	William	Dyer	DR-1	1927-02-08	-49.85	-128.57
2	622	dyer	rad	7.80	William	Dyer	DR-1	1927-02-10	-49.85	-128.57
3	622	dyer	sal	0.09	William	Dyer	DR-1	1927-02-10	-49.85	-128.57
4	844	roe	rad	11.25	Valentina	Roerich	DR-1	1932-03-22	-49.85	-128.57
5	734	pb	rad	8.41	Frank	Pabodie	DR-3	1939-01-07	-47.15	-126.72
6	734	pb	temp	-21.50	Frank	Pabodie	DR-3	1939-01-07	-47.15	-126.72
7	734	lake	sal	0.05	Anderson	Lake	DR-3	1939-01-07	-47.15	-126.72
8	735	pb	rad	7.22	Frank	Pabodie	DR-3	1930-01-12	-47.15	-126.72
9	751	pb	rad	4.35	Frank	Pabodie	DR-3	1930-02-26	-47.15	-126.72
10	751	pb	temp	-18.50	Frank	Pabodie	DR-3	1930-02-26	-47.15	-126.72
11	751	lake	sal	0.10	Anderson	Lake	DR-3	1930-02-26	-47.15	-126.72
12	752	lake	rad	2.19	Anderson	Lake	DR-3	NaN	-47.15	-126.72
13	752	lake	sal	0.09	Anderson	Lake	DR-3	NaN	-47.15	-126.72
14	752	lake	temp	-16.00	Anderson	Lake	DR-3	NaN	-47.15	-126.72
15	752	roe	sal	41.60	Valentina	Roerich	DR-3	NaN	-47.15	-126.72
16	837	lake	rad	1.46	Anderson	Lake	MSK-4	1932-01-14	-48.87	-123.40
17	837	lake	sal	0.21	Anderson	Lake	MSK-4	1932-01-14	-48.87	-123.40
18	837	roe	sal	22.50	Valentina	Roerich	MSK-4	1932-01-14	-48.87	-123.40

- 누락값 처리

- > 누락값 확인하기

- 누락값 : NaN으로 표기되어 있는 값

```
import numpy as np
print(np.NaN)
print(np.nan)
print(np.NAN)
```

Out : nan

nan

nan

• 누락값 처리

> 누락값 확인하기

- 누락값 : NaN으로 표기되어 있는 값

```
print(np.NaN == np.NaN)
print(np.NaN == None)
print(np.NaN == False)
print(np.NaN == 0)
```

Out : False

False

False

False

값 자체가 없기에 어떠한 값과 비교해도 똑같지 않다.

• 누락값 처리

> isnull, notnull 메서드로 누락값 확인하기

- isnull : 누락값이 있으면 True
- notnull : 누락값이 없으면 True

```
import pandas as pd
print(pd.isnull(np.NaN))
print(pd.isnull(None))
print(pd.notnull(np.NaN))
print(pd.notnull(False))
print(pd.notnull(0))
```

Out : True

True

False

True

True

• 누락값 처리

이걸로 뭘 하면 좋으려나. 아직까지는 잘 모르겠음.

> 누락값 개수 확인

- ebola 데이터셋 가져오기

```
ebola = pd.read_csv('data/country_timeseries.csv')
ebola.head()
```

Out :

	Date	Day	Cases_Guinea	Cases_Liberia	Cases_SierraLeone	Cases_Nigeria	Cases_Senegal	Cases_UnitedStates	Cases_Spain	Cases_Mali
0	1/5/2015	289	2776.0	NaN	10030.0	NaN	NaN	NaN	NaN	NaN
1	1/4/2015	288	2775.0	NaN	9780.0	NaN	NaN	NaN	NaN	NaN
2	1/3/2015	287	2769.0	8166.0	9722.0	NaN	NaN	NaN	NaN	NaN
3	1/2/2015	286	NaN	8157.0	NaN	NaN	NaN	NaN	NaN	NaN
4	12/31/2014	284	2730.0	8115.0	9633.0	NaN	NaN	NaN	NaN	NaN

Deaths_Guinea	Deaths_Liberia	Deaths_SierraLeone	Deaths_Nigeria	Deaths_Senegal	Deaths_UnitedStates	Deaths_Spain	Deaths_Mali
1786.0	NaN	2977.0	NaN	NaN	NaN	NaN	NaN
1781.0	NaN	2943.0	NaN	NaN	NaN	NaN	NaN
1767.0	3496.0	2915.0	NaN	NaN	NaN	NaN	NaN
NaN	3496.0	NaN	NaN	NaN	NaN	NaN	NaN
1739.0	3471.0	2827.0	NaN	NaN	NaN	NaN	NaN

• 누락값 처리

- > 누락값 개수 확인
 - count() 메서드 사용

```
ebola.count()
```

Out :

Date	122
Day	122
Cases_Guinea	93
Cases_Liberia	83
Cases_SierraLeone	87
Cases_Nigeria	38
Cases_Senegal	25
Cases_UnitedStates	18
Cases_Spain	16
Cases_Mali	12
Deaths_Guinea	92
Deaths_Liberia	81
Deaths_SierraLeone	87
Deaths_Nigeria	38
Deaths_Senegal	22
Deaths_UnitedStates	18
Deaths_Spain	16
Deaths_Mali	12
dtype: int64	

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 122 entries, 0 to 121  
Data columns (total 18 columns):  
#   Column      Non-Null Count  Dtype  
---  ---  
0   Date        122 non-null    object  
1   Day         122 non-null    int64  
2   Cases_Guinea 93 non-null     float64  
3   Cases_Liberia 83 non-null     float64  
4   Cases_SierraLeone 87 non-null     float64  
5   Cases_Nigeria 38 non-null     float64  
6   Cases_Senegal 25 non-null     float64  
7   Cases_UnitedStates 18 non-null     float64  
8   Cases_Spain  16 non-null     float64  
9   Cases_Mali   12 non-null     float64  
10  Deaths_Guinea 92 non-null     float64  
11  Deaths_Liberia 81 non-null     float64  
12  Deaths_SierraLeone 87 non-null     float64  
13  Deaths_Nigeria 38 non-null     float64  
14  Deaths_Senegal 22 non-null     float64  
15  Deaths_UnitedStates 18 non-null     float64  
16  Deaths_Spain  16 non-null     float64  
17  Deaths_Mali   12 non-null     float64  
dtypes: float64(16), int64(1), object(1)  
memory usage: 17.3+ KB
```

• 누락값 처리

> 누락값 개수 확인

- count() 메서드 사용

```
ebola.shape[0] - ebola.count()
```

```
Out : Date          0
      Day           0
      Cases_Guinea  29
      Cases_Liberia 39
      Cases_SierraLeone 35
      Cases_Nigeria 84
      Cases_Senegal  97
      Cases_UnitedStates 104
      Cases_Spain    106
      Cases_Mali     110
      Deaths_Guinea  30
      Deaths_Liberia 41
      Deaths_SierraLeone 35
      Deaths_Nigeria 84
      Deaths_Senegal 100
      Deaths_UnitedStates 104
      Deaths_Spain   106
      Deaths_Mali    110
      dtype: int64
```

[illegible]

• 누락값 처리

> 누락값 개수 확인

- isnull() 과 sum() 메서드 함께 사용

```
ebola.isnull().sum()
```

```
Out : Date          0
      Day           0
      Cases_Guinea  29
      Cases_Liberia 39
      Cases_SierraLeone 35
      Cases_Nigeria 84
      Cases_Senegal  97
      Cases_UnitedStates 104
      Cases_Spain    106
      Cases_Mali     110
      Deaths_Guinea  30
      Deaths_Liberia 41
      Deaths_SierraLeone 35
      Deaths_Nigeria 84
      Deaths_Senegal 100
      Deaths_UnitedStates 104
      Deaths_Spain   106
      Deaths_Mali    110
      dtype: int64
```

• 누락값 처리

> 누락값 개수 확인

- value_counts() 메서드 사용 - 시리즈에 사용

```
ebola['Cases_Guinea'].value_counts(dropna = False)
```

Out :

NaN	29
86.0	3
495.0	2
112.0	2
390.0	2
..	..
1199.0	1
1298.0	1
1350.0	1
1472.0	1
49.0	1

• 누락값 처리

군데군데 비어있는 구멍을 메꿔서 써야한다.
왜냐하면 데이터가 귀하기 때문이다.

- > 누락값 변경하기
 - fillna 메서드

```
ebola.fillna(0)
```

Out :

Date	Day	Cases_Guinea	Cases_Liberia	Cases_SierraLeone
1/5/2015	289	2776.0	0.0	10030.0
1/4/2015	288	2775.0	0.0	9780.0
1/3/2015	287	2769.0	8166.0	9722.0
1/2/2015	286	0.0	8157.0	0.0
12/31/2014	284	2730.0	8115.0	9633.0

• 누락값 처리

데이터를 어떻게 채울 것인가: 이전 값으로 채우기

> 누락값 변경하기

- fillna 메서드 - ffill

값을 NaN으로 뒤야하는 거 아닌가? 왜 누락 값을 채워야 하는걸까?
평균 계산하거나 할 때 없는값을 포함하면 안되는 거 아닌가..

```
ebola.fillna(method = 'ffill')
```

누락 값을 채우는 이유
-> 실제 정확한 값과 오차를 줄이면서 값을 채우기
위함이다!

Out :

Date	Day	Cases_Guinea	Cases_Liberia	Cases_SierraLeone
1/5/2015	289	2776.0	NaN	10030.0
1/4/2015	288	2775.0	NaN	9780.0
1/3/2015	287	2769.0	8166.0	9722.0
1/2/2015	286	2769.0	8157.0	9722.0
12/31/2014	284	2730.0	8115.0	9633.0

앞 데이터를 똑같이 채워주기에 앞에 데이터가 없으면 값을 채우지 못한다.

• 누락값 처리

> 누락값 변경하기

- fillna 메서드 - bfill

```
ebola.fillna(method = 'bfill')
```

Out :

Date	Day	Cases_Guinea	Cases_Liberia	Cases_SierraLeone
1/5/2015	289	2776.0	8166.0	10030.0
1/4/2015	288	2775.0	8166.0	9780.0
1/3/2015	287	2769.0	8166.0	9722.0
1/2/2015	286	2730.0	8157.0	9633.0
12/31/2014	284	2730.0	8115.0	9633.0
3/27/2014	5	103.0	8.0	6.0
3/26/2014	4	86.0	NaN	NaN
3/25/2014	3	86.0	NaN	NaN
3/24/2014	2	86.0	NaN	NaN
3/22/2014	0	49.0	NaN	NaN

• 누락값 처리

> 누락값 변경하기

- interpolate 메서드

```
ebola.interpolate()
```

Out :

	Date	Day	Cases_Guinea	Cases_Liberia	Cases_SierraLeone
0	1/5/2015	289	2776.0	NaN	10030.0
1	1/4/2015	288	2775.0	NaN	9780.0
2	1/3/2015	287	2769.0	8166.0	9722.0
3	1/2/2015	286	2749.5	8157.0	9677.5
4	12/31/2014	284	2730.0	8115.0	9633.0
...	...	...	...	...	...
117	3/27/2014	5	103.0	8.0	6.0
118	3/26/2014	4	86.0	8.0	6.0
119	3/25/2014	3	86.0	8.0	6.0
120	3/24/2014	2	86.0	8.0	6.0
121	3/22/2014	0	49.0	8.0	6.0

• 누락값 처리

- > 누락값 삭제하기
 - dropna 메서드

```
ebola.dropna()
```

Out :

	Date	Day	Cases_Guinea	Cases_Liberia	Cases_SierraLeone	Cases_Nigeria
19	11/18/2014	241	2047.0	7082.0	6190.0	20.0

```
ebola.dropna(axis = 1)
```

Out :

	Date	Day
0	1/5/2015	289
1	1/4/2015	288
2	1/3/2015	287
3	1/2/2015	286
4	12/31/2014	284

• 누락값 처리

> 누락값 일부분만 삭제하기

- `dropna(subset = ['열 이름'])`

```
ebola.dropna(subset=['Cases_Guinea'])
```

Out :

	Date	Day	Cases_Guinea	Cases_Liberia	Cases_SierraLeone
0	1/5/2015	289	2776.0	NaN	10030.0
1	1/4/2015	288	2775.0	NaN	9780.0
2	1/3/2015	287	2769.0	8166.0	9722.0
4	12/31/2014	284	2730.0	8115.0	9633.0
5	12/28/2014	281	2706.0	8018.0	9446.0

- 간단한 함수 만들기

- > 제곱 함수와 n 제곱 함수 만들기
 - 제곱 사용자 함수

```
def my_sq(x):  
    return x ** 2
```

- n 제곱 사용자 함수

```
def my_exp(x, n):  
    return x ** n
```

- 사용자 함수 사용

```
print(my_sq(4))  
print(my_exp(3, 4))
```

Out : 16

81

• 간단한 함수 만들기

- > apply 메소드 사용하기 - 기초
 - 데이터프레임 준비

```
import pandas as pd
```

```
df = pd.DataFrame({'a': [10, 20, 30], 'b': [20, 30, 40]})  
df
```

Out :

	a	b
0	10	20
1	20	30
2	30	40

- apply 메소드 사용하기

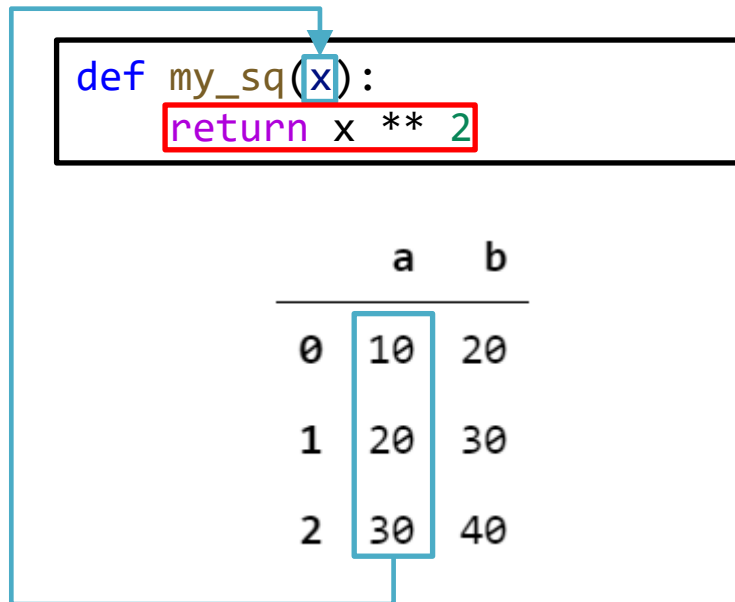
- > 시리즈에 apply 메소드 사용
 - 제공 함수 (my_sq) 적용 : 인자 1개

```
sq = df['a'].apply(my_sq)  
print(sq)
```

Out :

0	100
1	400
2	900

Name: a, dtype: int64



• apply 메소드 사용하기

> 시리즈에 apply 메소드 사용

- n 제공 함수 (my_exp) 적용 : 인자 2개

```
ex = df['a'].apply(my_exp, n=2)  
print(ex)
```

Out :

0	100
1	400
2	900

Name: a, dtype: int64

```
def my_exp(x, n):  
    return x ** n
```

	a	b
0	10	20
1	20	30
2	30	40

- apply 메소드 사용하기

> apply 메소드 활용법

- 타이타닉 데이터

```
titanic = pd.read_csv('data/titanic.csv')
titanic.head()
```

Out :

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S

- apply 메소드 사용하기

- > apply 메소드 활용법

- 타이타닉 데이터

```
titanic.info()
```

Out : <class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
Column Non-Null Count Dtype
--- -
0 PassengerId 891 non-null int64
1 Survived 891 non-null int64
2 Pclass 891 non-null int64
3 Name 891 non-null object
4 Sex 891 non-null object
5 Age 714 non-null float64
6 SibSp 891 non-null int64
7 Parch 891 non-null int64
8 Ticket 891 non-null object
9 Fare 891 non-null float64
10 Cabin 204 non-null object
11 Embarked 889 non-null object
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB

- apply 메소드 사용하기

- > apply 메소드 활용법

- apply 함수를 사용하여 값 변경하기

```
def trans(x):  
    # 'male', 'female'  
    if x == 'male':  
        a = 'M'  
    else:  
        a = 'F'  
    return a  
titanic['Sex'].apply(trans)
```

Out :

```
0      M  
1      F  
2      F  
3      F  
4      M  
...  
886     M  
887     F  
888     F  
889     M  
890     M  
Name: Sex, Length: 891, dtype: object
```

• 데이터 집계

> groupby 메서드로 평균값 구하기

- 갭마인더 데이터

```
import pandas as pd
df = pd.read_csv('data/gapminder.tsv', sep='\t')
```

- year 열을 기준으로 그룹화하고 lifeExp 평균 구하기

```
avg_lifeexp_by_year = df.groupby('year')['lifeExp'].mean()
avg_lifeexp_by_year
```

Out :

year	
1952	49.057620
1957	51.507401
1962	53.609249
1967	55.678290
1972	57.647386
1977	59.570157
1982	61.533197
1987	63.212613
1992	64.160338
1997	65.014676
2002	65.694923
2007	67.007423

Name: lifeExp, dtype: float64

- 데이터 집계

> groupby 메서드의 집계 메서드

메소드	설명
count	누락값을 제외한 데이터 수를 반환
size	누락값을 포함한 데이터 수를 반환
mean	평균값 반환
std	표준편차 반환
min	최소값 반환
quantile(q=0.25 / 0.50 / 0.75)	백분위수 25% / 50% / 75%
max	최대값 반환
sum	전체 합 반환
var	분산 반환
describe	통계 요약 정보 반환
first	첫번째 행 반환
last	마지막 행 반환
nth	n번째 행 반환

- 데이터 집계

> groupby에 사용자 함수 적용

- agg() : apply와 유사

```
def my_mean(values):  
    n = len(values)  
    sum = 0  
    for value in values:  
        sum += value  
    return sum / n
```

- 데이터 집계

> groupby에 사용자 함수 적용

- agg() : apply와 유사

```
agg_my_mean = df.groupby('year')['lifeExp'].agg(my_mean)
print(agg_my_mean)
```

Out :

year	
1952	49.057620
1957	51.507401
1962	53.609249
1967	55.678290
1972	57.647386
1977	59.570157
1982	61.533197
1987	63.212613
1992	64.160338
1997	65.014676
2002	65.694923
2007	67.007423

Name: lifeExp, dtype: float64

- 데이터 집계

> groupby에 사용자 함수 적용

- 2개의 인자 사용법

```
def my_mean_diff(values, diff_value):  
    n = len(values)  
    sum = 0  
    for value in values:  
        sum += value  
    mean = sum / n  
    return mean - diff_value
```

```
global_mean = df['lifeExp'].mean()  
print(global_mean)
```

Out : 59.47443936619714

• 데이터 집계

> groupby에 사용자 함수 적용

- 2개의 인자 사용법

```
agg_mean_diff = df.groupby('year')['lifeExp'].agg(my_mean_diff,  
                                                    diff_value=global_mean)
```

```
agg_mean_diff
```

Out :

year	
1952	-10.416820
1957	-7.967038
1962	-5.865190
1967	-3.796150
1972	-1.827053
1977	0.095718
1982	2.058758
1987	3.738173
1992	4.685899
1997	5.540237
2002	6.220483
2007	7.532983

Name: lifeExp, dtype: float64

• 데이터 집계

> 여러 개의 집계 메서드 한번에 적용

- mean, std 적용

```
df.groupby('year')['lifeExp'].agg(['mean', 'std'])
```

Out :

	mean	std
year		
1952	49.057620	12.225956
1957	51.507401	12.231286
1962	53.609249	12.097245
1967	55.678290	11.718858
1972	57.647386	11.381953
1977	59.570157	11.227229
1982	61.533197	10.770618
1987	63.212613	10.556285
1992	64.160338	11.227380
1997	65.014676	11.559439
2002	65.694923	12.279823
2007	67.007423	12.073021

• 데이터 집계

> 여러 개의 집계 메서드 한번에 적용

- 넘파이 함수 np.mean, np.std, 사용자 함수 적용

```
import numpy as np
df.groupby('year')['lifeExp'].agg([np.mean, np.std, my_mean])
```

Out :

	mean	std	my_mean
year			
1952	49.057620	12.225956	49.057620
1957	51.507401	12.231286	51.507401
1962	53.609249	12.097245	53.609249
1967	55.678290	11.718858	55.678290
1972	57.647386	11.381953	57.647386
1977	59.570157	11.227229	59.570157
1982	61.533197	10.770618	61.533197
1987	63.212613	10.556285	63.212613
1992	64.160338	11.227380	64.160338
1997	65.014676	11.559439	65.014676
2002	65.694923	12.279823	65.694923
2007	67.007423	12.073021	67.007423

• 데이터 집계

> 그룹 오브젝트

- 그룹 오브젝트의 속성 확인하기

```
grouped = df.groupby('year')  
grouped
```

Out : <pandas.core.groupby.generic.DataFrameGroupBy object at 0x000001F947FA54C0>

- 그룹 인덱스 확인

```
grouped.groups
```

Out : {1957: [937], 1972: [1456], 1977: [1493], 1992: [776, 1544],
1997: [1545], 2002: [1342, 106], 2007: [251, 215]}

• 데이터 집계

> 그룹 오브젝트

- 그룹명을 지정하여 데이터 추출

```
grouped.get_group(2002)
```

Out :

	country	continent	year	lifeExp	pop	gdpPercap	fill_lifeExp
1342	Serbia	Europe	2002	73.213	10111559	7236.075251	73.213
106	Bangladesh	Asia	2002	NaN	135656790	1136.390430	73.213

- 각 그룹 정보 추출

```
for group in grouped:
    print(group)
```

Out :

```
(1957,      country continent  year  lifeExp      pop      gdpPercap  fill_lifeExp
937  Malaysia      Asia 1957   52.102  7739235  1810.066992      52.102)
(1972,      country continent  year  lifeExp      pop      gdpPercap  fill_lifeExp
1456  Swaziland   Africa 1972     NaN   480105   3364.836625      NaN)
```

- 수많은 데이터를 한 눈에 파악하는 방법

- > 그래프

- Histogram
- Scatter
- Density plot
- Bar plot
- Box plot
- Line chart
- ...

- > 통계량

- Min, Max, Sum
- Mean, Std
- 사분위수
- 검정 통계량
- P-value
- ...

- 수많은 데이터를 한 눈에 파악하는 방법

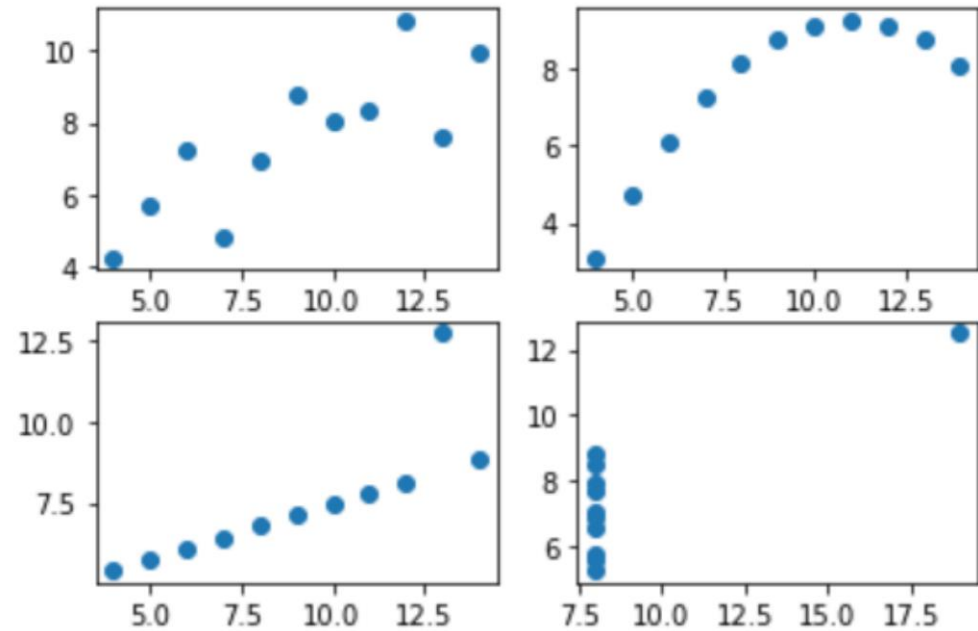
> 데이터 요약의 목적

- 정보화시대에 데이터는 비즈니스를 담고 있다.
- 시각화, 통계량을 통해 방대한 데이터를 요약하여
- 비즈니스의 인사이트를 파악할 수 있다.
- 그래프, 통계량을 통한 데이터 요약은 정보의 손실이 발생함.

• 데이터 시각화가 필요한 이유

> 앤스콤 4분할 그래프

- 데이터 시각화의 전형적인 사례
- 시각화 하지 않고 수치만 확인할 때 발생할 수 있는 문제점을 보여줌
- 4개의 데이터 그룹으로 구성되며, 평균, 분산, 상관관계, 회귀선이 모두 같은 데이터의 특성이 있다.
 - 4개의 특성이 같으므로 4개의 데이터는 비슷한 유형이다 라는 착각
- 앤스콤 그래프



- 데이터 시각화가 필요한 이유

- > 앤스콤 데이터 집합 불러오기
 - seaborn 라이브러리 데이터셋

```
import seaborn as sns
anscombe = sns.load_dataset('anscombe')
anscombe.head()
```

Out :

	dataset	x	y
0	I	10.0	8.04
1	I	8.0	6.95
2	I	13.0	7.58
3	I	9.0	8.81
4	I	11.0	8.33

- 데이터 시각화가 필요한 이유

- > 데이터 살펴보기

- 데이터셋 별로 평균과 분산 구하기

```
anscombe.groupby('dataset').mean()
```

Out :

	x	y
dataset		
I	9.0	7.500909
II	9.0	7.500909
III	9.0	7.500000
IV	9.0	7.500909

```
anscombe.groupby('dataset').var()
```

Out :

	x	y
dataset		
I	11.0	4.127269
II	11.0	4.127629
III	11.0	4.122620
IV	11.0	4.123249

- 데이터 시각화가 필요한 이유

- > 데이터 살펴보기

- 데이터셋 별로 상관관계 구하기 - corr() 함수

```
anscombe.groupby('dataset').corr()
```

Out :

		x	y
dataset	I	x	1.000000
	y	0.816421	1.000000
II	x	1.000000	0.816237
	y	0.816237	1.000000
III	x	1.000000	0.816287
	y	0.816287	1.000000
IV	x	1.000000	0.816521
	y	0.816521	1.000000

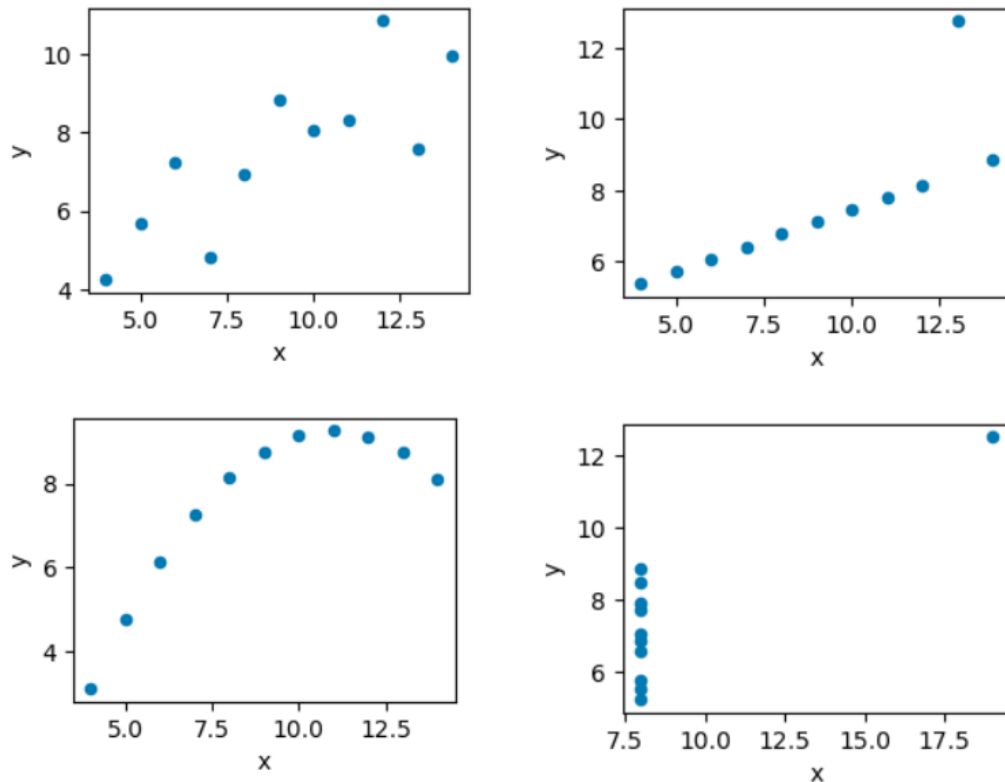
- 데이터 시각화가 필요한 이유

- > 그래프 그리기

- 데이터셋 별로 그래프 그리기

```
anscombe.groupby('dataset').plot('x', 'y', kind='scatter')
```

Out :



- Matplotlib

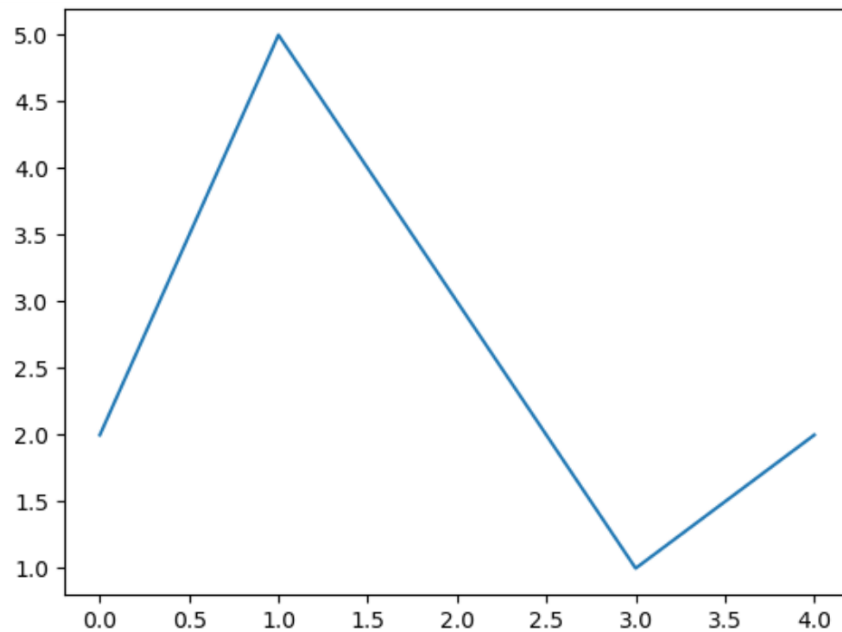
- > 기본기

```
import matplotlib.pyplot as plt
```

```
plt.plot([2, 5, 3, 1, 2])
```

```
plt.show()
```

Out :



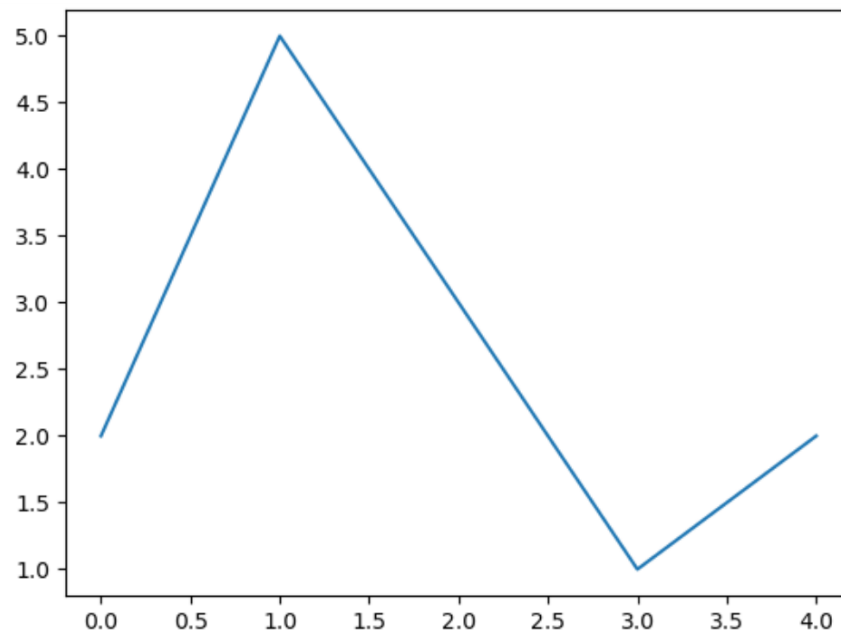
- Matplotlib

- > 이미지 저장하기

```
plt.plot([2, 5, 3, 1, 2])  
plt.savefig('a')
```

Out :

  a.png

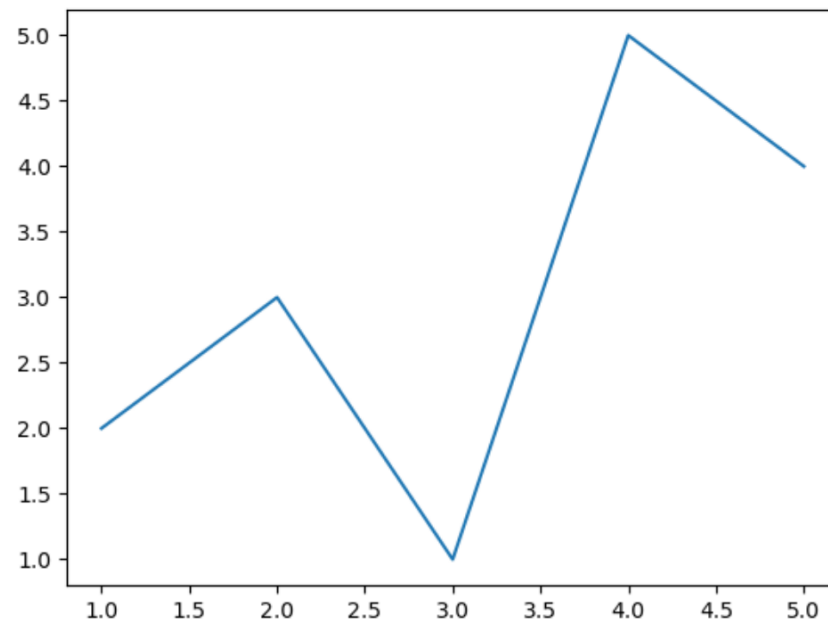


- Matplotlib

> X, y축 지정

```
X = [1, 2, 3, 4, 5]  
y = [2, 3, 1, 5, 4]  
plt.plot(X, y)
```

Out :

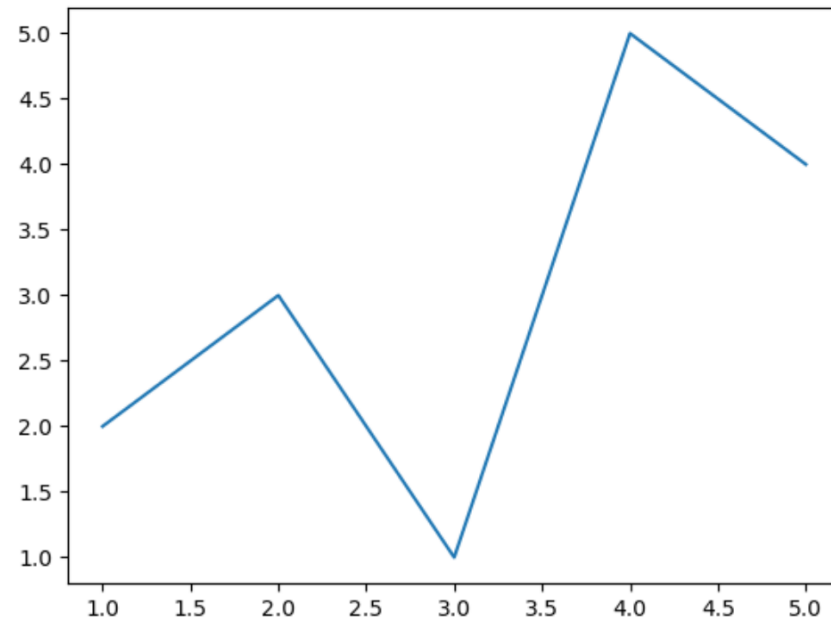


- Matplotlib

> X, y축 지정

```
import pandas as pd
df = pd.DataFrame({'X':[1, 2, 3, 4, 5],
                  'y':[2, 3, 1, 5, 4]})
plt.plot('X', 'y', data=df)
```

Out :

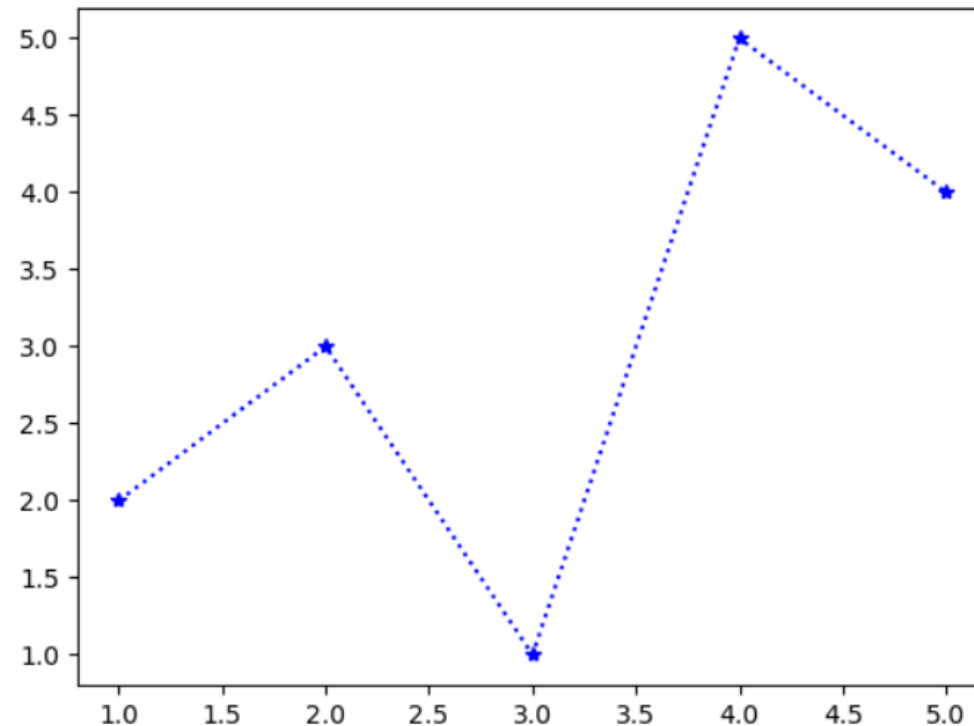


- Matplotlib

> 라인 스타일 조정

```
plt.plot('X', 'y', 'b*:', data=df)
```

Out :



• Matplotlib

> 라인 스타일 조정

character	color
'b'	blue
'g'	green
'r'	red
'c'	cyan
'm'	magenta
'y'	yellow
'k'	black
'w'	white

character	description
'_'	solid line style
'--'	dashed line style
'-.'	dash-dot line style
':'	dotted line style

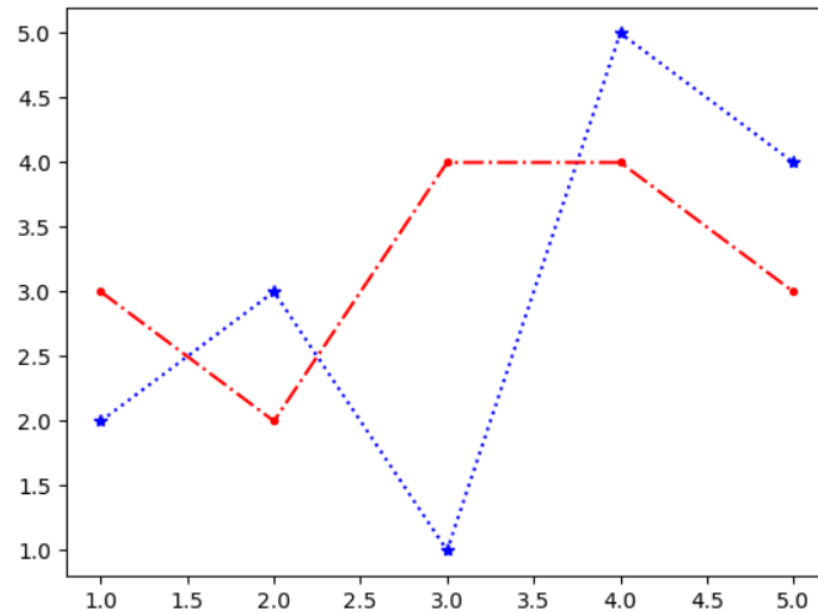
character	description
'.'	point marker
','	pixel marker
'o'	circle marker
'v'	triangle_down marker
'^'	triangle_up marker
'<'	triangle_left marker
'>'	triangle_right marker
'1'	tri_down marker
'2'	tri_up marker
'3'	tri_left marker
'4'	tri_right marker
's'	square marker
'p'	pentagon marker
'*'	star marker
'h'	hexagon1 marker
'H'	hexagon2 marker
'+'	plus marker
'x'	x marker
'D'	diamond marker
'd'	thin_diamond marker
' '	vline marker
'_'	hline marker

- Matplotlib

- > 그래프 겹치기

```
df['y2'] = [3, 2, 4, 4, 3]  
plt.plot('X', 'y', 'b*:', data=df)  
plt.plot('X', 'y2', 'r.-.', data=df)
```

Out :

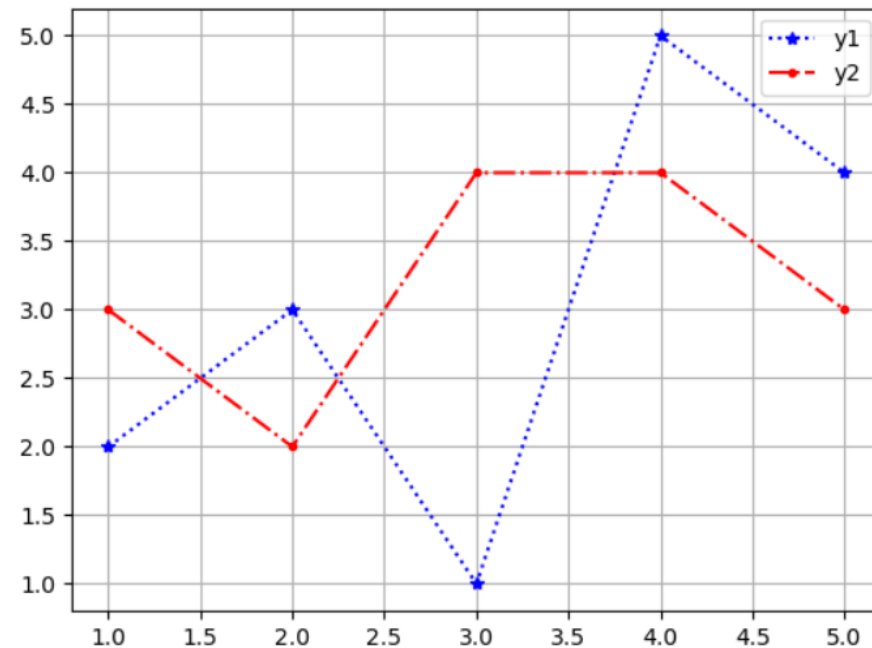


- Matplotlib

> 범례, 격자 그래프

```
plt.plot('X', 'y', 'b*:', data=df, label='y1')  
plt.plot('X', 'y2', 'r.-.', data=df, label='y2')  
plt.grid()  
plt.legend()
```

Out :

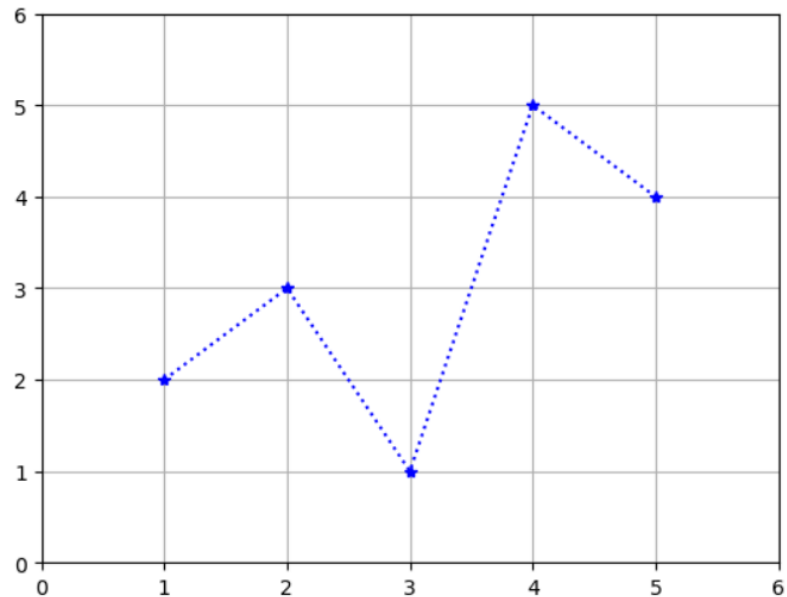


- Matplotlib

- > 축 범위 지정

```
plt.plot('X', 'y', 'b*:', data=df)  
plt.xlim(0, 6)  
plt.ylim(0, 6)  
plt.grid()
```

Out :

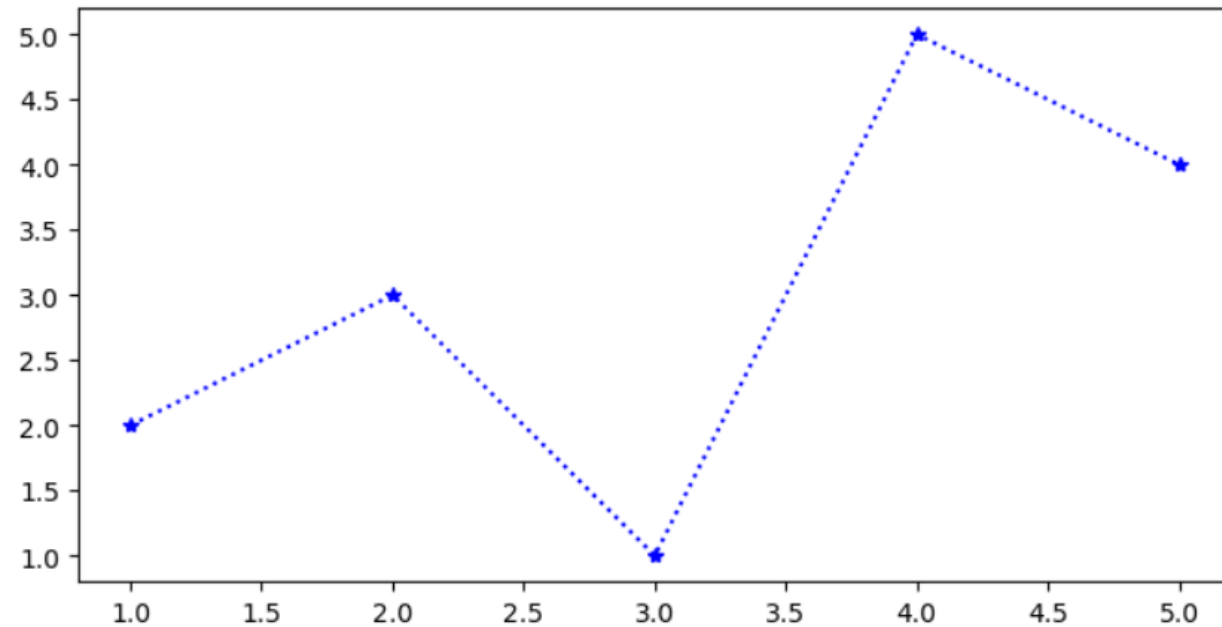


- Matplotlib

- > 그래프 크기 조정

```
plt.figure(figsize = (8, 4))  
plt.plot('X', 'y', 'b*:', data=df)
```

Out :



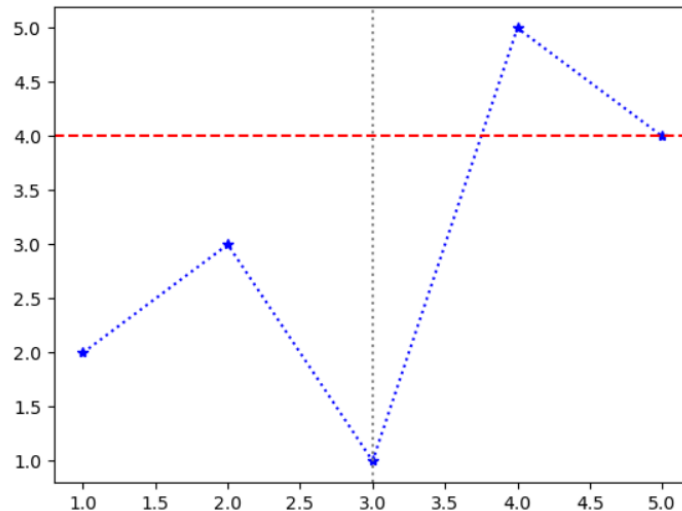
• Matplotlib

> 수평선, 수직선 추가하기

- `axhline(y축값, color, linestyle)`
- `axvline(x축값, color, linestyle)`

```
plt.plot('X', 'y', 'b*:', data=df)  
plt.axhline(4, color='red', linestyle = '--')  
plt.axvline(3, color='grey', linestyle = ':')
```

Out :

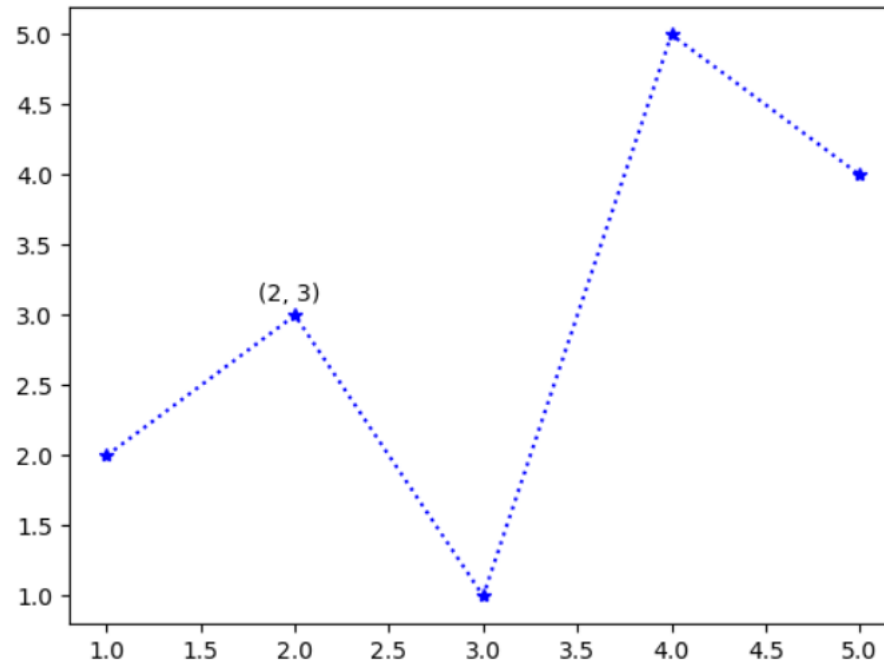


- Matplotlib

> 그래프에 텍스트 추가

```
plt.plot('X', 'y', 'b*:', data=df)  
plt.text(1.8, 3.1, '(2, 3)')
```

Out :

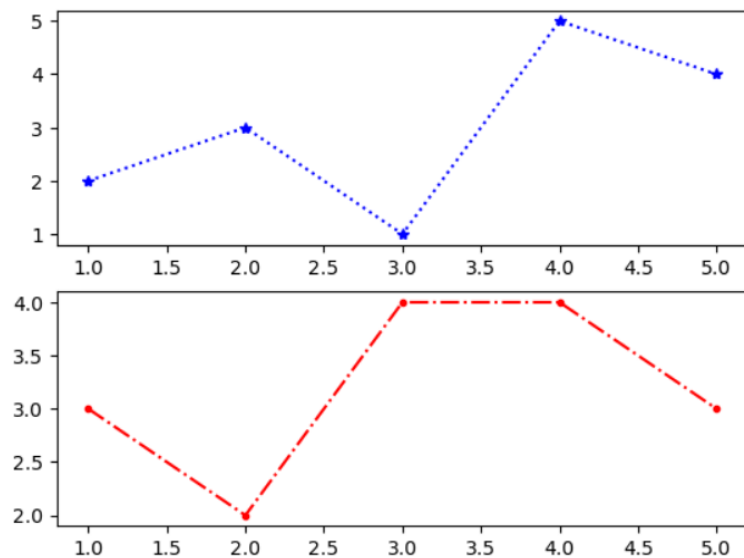


- Matplotlib

- > 그래프 나누기

```
plt.subplot(2, 1, 1)  
plt.plot('X', 'y', 'b*:', data=df)  
plt.subplot(2, 1, 2)  
plt.plot('X', 'y2', 'r-.', data=df)
```

Out :



- Matplotlib

- > 그래프 나누기

- subplot(rows, cols, index)

subplot(2,1,1)

subplot(2,1,2)

subplot(1,2,1)

subplot(1,2,2)

subplot(2,2,1)

subplot(2,2,2)

subplot(2,2,3)

subplot(2,2,4)

• Matplotlib

> 그래프 나누기

- title() - 그래프의 title
- subplot() - 그래프의 큰 title
- tight_layout() - 그래프 간격 조정

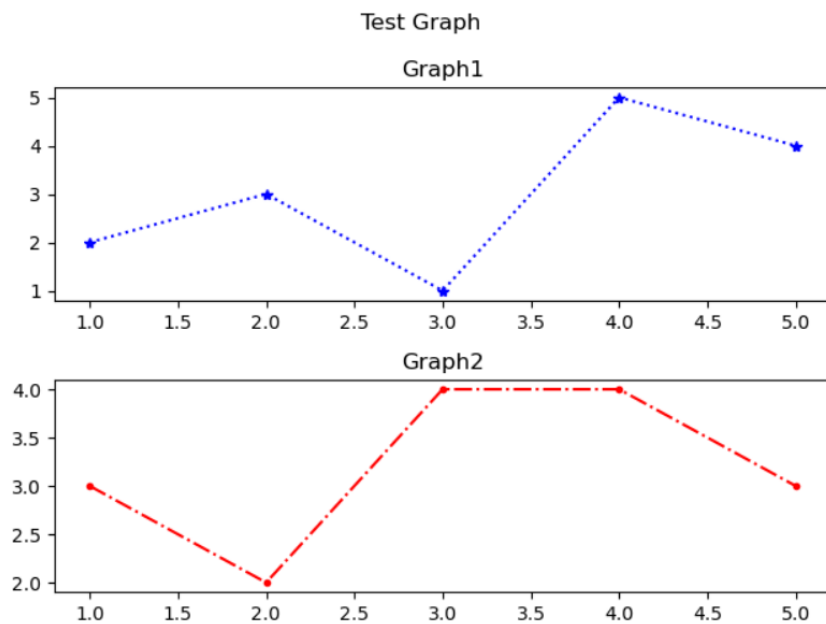
```
plt.subplot(2, 1, 1)
plt.plot('X', 'y', 'b*:', data=df)
plt.title('Graph1')
plt.subplot(2, 1, 2)
plt.plot('X', 'y2', 'r.-.', data=df)
plt.title('Graph2')
plt.suptitle('Test Graph')
plt.tight_layout()
```

• Matplotlib

> 그래프 나누기

- title() - 그래프의 title
- subtitle() - 그래프의 큰 title
- tight_layout() - 그래프 간격 조정

Out :

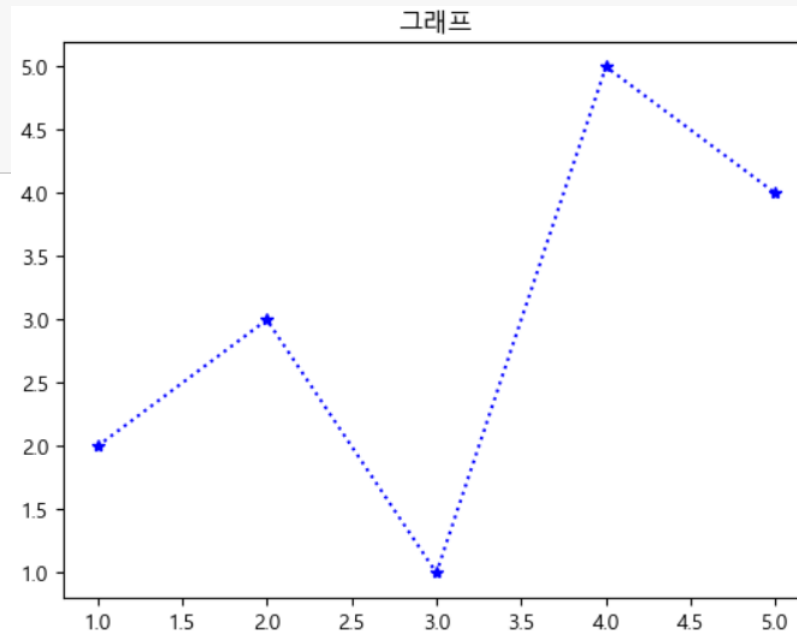


- Matplotlib

- > 한글 표현하기

```
from matplotlib import font_manager, rc
font = 'C:/Windows/Fonts/Malgun.ttf'
font_name = font_manager.FontProperties(fname=font).get_name()
rc('font', family=font_name)
plt.plot('X', 'y', 'b*:', data=df)
plt.title('그래프')
```

Out :



- 다양한 그래프

> 데이터 준비

- seaborn 라이브러리의 tips 데이터셋

```
tips = sns.load_dataset('tips')
tips
```

Out :

	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4
...	...	...	...	...	...	...	...
239	29.03	5.92	Male	No	Sat	Dinner	3
240	27.18	2.00	Female	Yes	Sat	Dinner	2
241	22.67	2.00	Male	Yes	Sat	Dinner	2
242	17.82	1.75	Male	No	Sat	Dinner	2
243	18.78	3.00	Female	No	Thur	Dinner	2

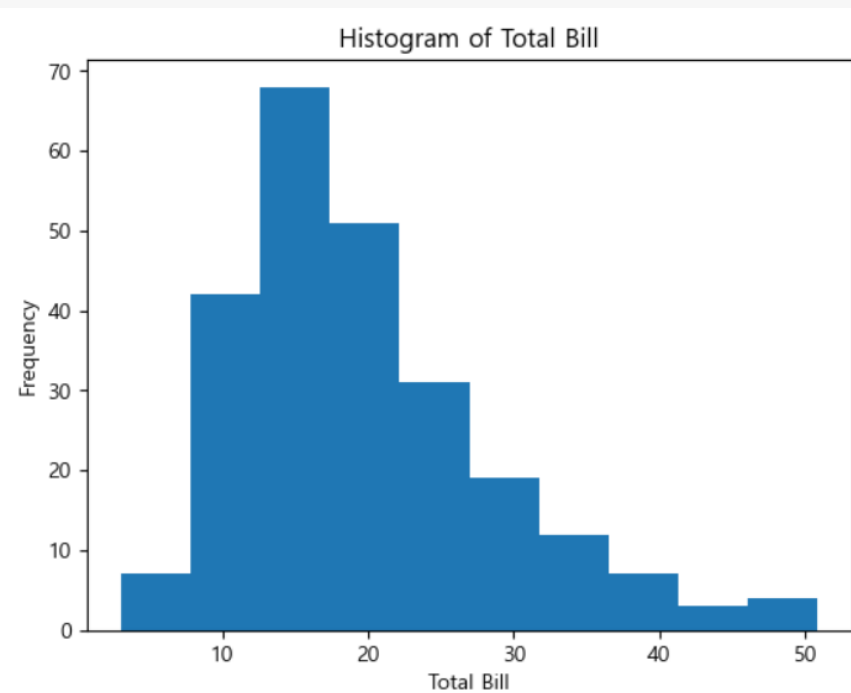
• 다양한 그래프

> 히스토그램

- 숫자형 데이터의 분포와 빈도를 표현

```
hist = plt.hist(tips['total_bill'], bins=10)
plt.title('Histogram of Total Bill')
plt.xlabel('Total Bill')
plt.ylabel('Frequency')
```

Out :



• 다양한 그래프

> 히스토그램

- 숫자형 데이터의 분포와 빈도를 표현

```
print('빈도수 :', hist[0])  
print('구간값 :', hist[1])
```

Out :

빈도수 : [7. 42. 68. 51. 31. 19. 12. 7. 3. 4.]

**구간값 : [3.07 7.844 12.618 17.392 22.166 26.94
31.714 36.488 41.262 46.036 50.81]**

• 다양한 그래프

> 박스 그래프

- 집단 간의 분포 차이를 표현

```
series = []  
for group in tips.groupby('sex')['total_bill']:  
    series.append(group[1])  
series
```

Out :

1	10.34	0	16.99
2	21.01	4	24.59
3	23.68	11	35.26
5	25.29	14	14.83
6	8.77	16	10.33
	...		...
236	12.60	226	10.09
237	32.83	229	22.12
239	29.03	238	35.83
241	22.67	240	27.18
242	17.82	243	18.78

Name: total_bill, Length: 157, dtype: float64, Name: total_bill, Length: 87, dtype: float64]

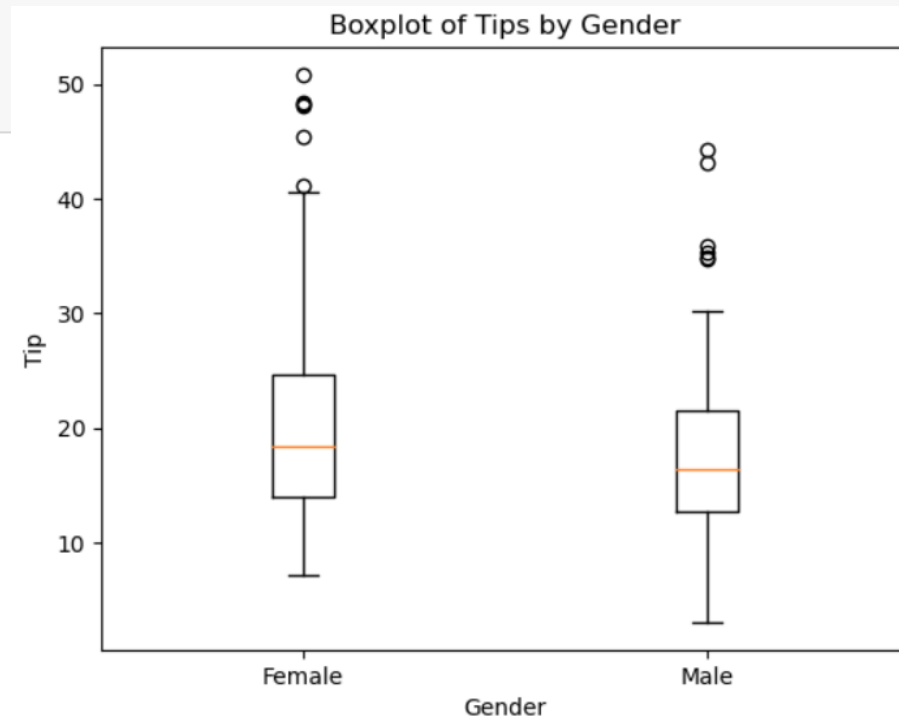
• 다양한 그래프

> 박스 그래프

- 집단 간의 분포 차이를 표현

```
box1 = plt.boxplot(series, labels=['Female', 'Male'])  
plt.title('Boxplot of Tips by Gender')  
plt.xlabel('Gender')  
plt.ylabel('Tip')
```

Out :



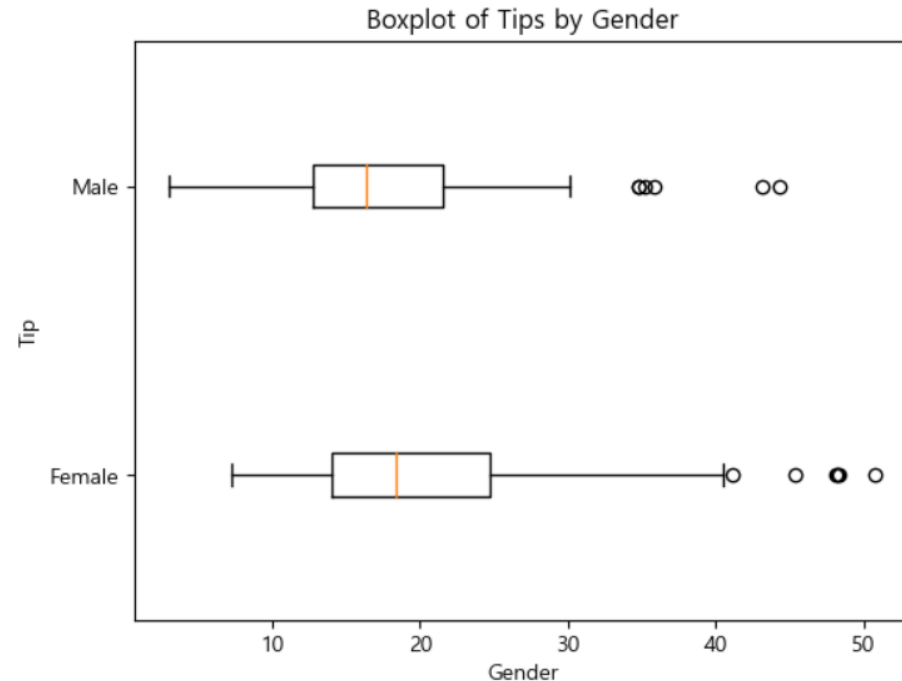
• 다양한 그래프

> 박스 그래프

- vert를 통해 가로로 그리기 가능

```
box2 = plt.boxplot(series, labels=['Female', 'Male'], vert = False)
plt.title('Boxplot of Tips by Gender')
plt.xlabel('Gender')
plt.ylabel('Tip')
```

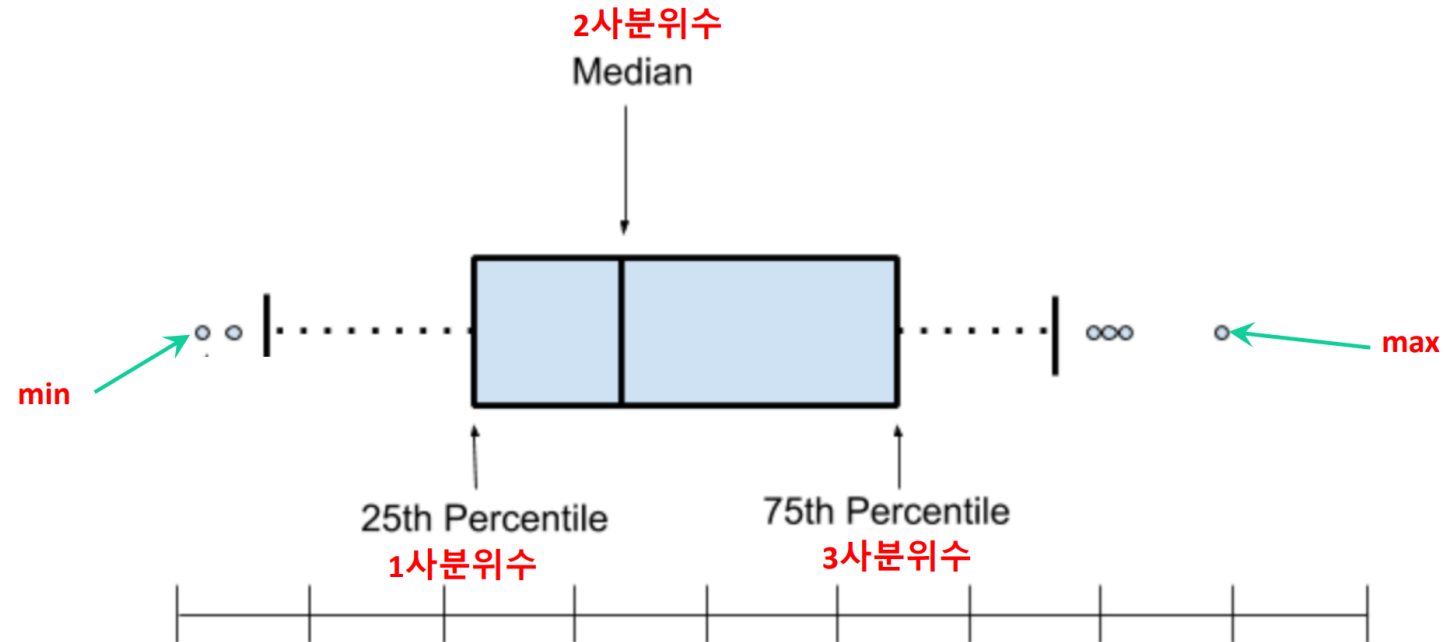
Out :



• 다양한 그래프

> 박스 그래프

- 집단 간의 분포 차이를 표현



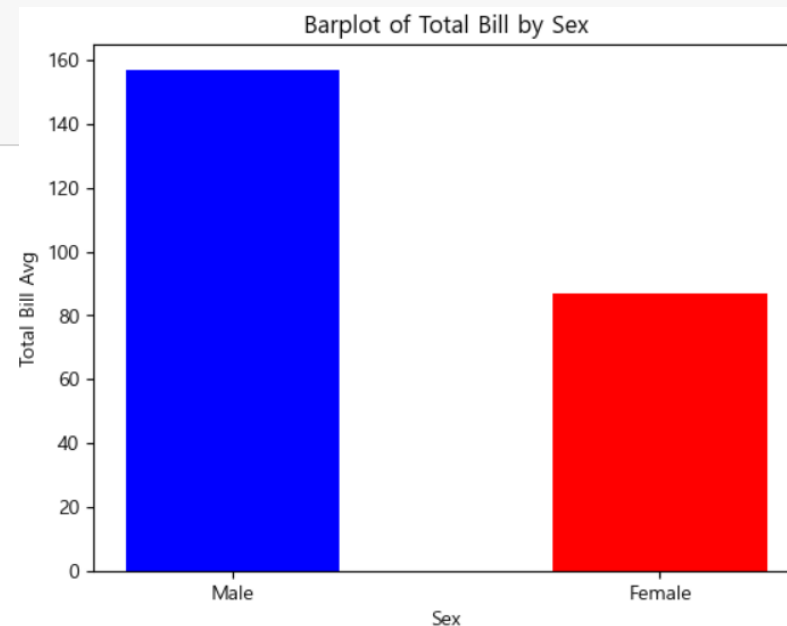
• 다양한 그래프

> 막대 그래프

- 집단 간의 차이를 표현

```
s_tips = tips['sex'].value_counts()  
plt.bar(s_tips.index, s_tips.values, color = ['b', 'r'], width = .5)  
plt.title('Barplot of Total Bill by Sex')  
plt.xlabel('Sex')  
plt.ylabel('Total Bill Count')
```

Out :



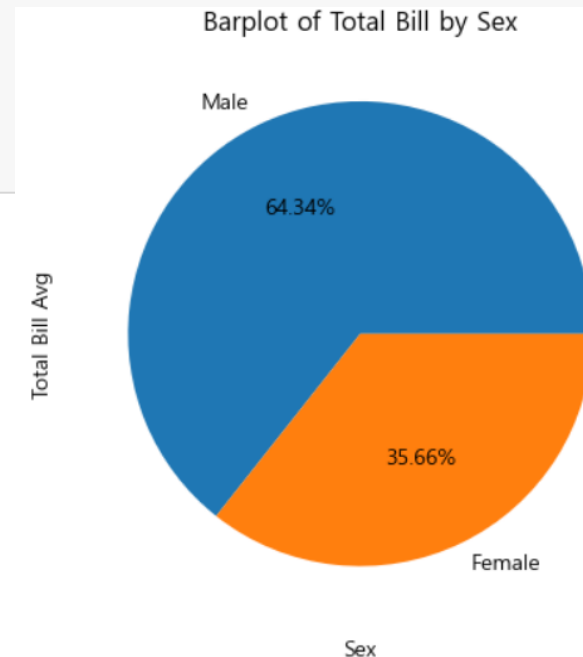
• 다양한 그래프

> 막대 그래프

- 집단 간의 비율을 표현

```
s_tips = tips['sex'].value_counts()
plt.pie(s_tips.values, labels = s_tips.index, autopct = '%.2f%%')
plt.title('Pie chart of Total Bill by Sex')
plt.xlabel('Sex')
plt.ylabel('Total Bill Ratio')
```

Out :



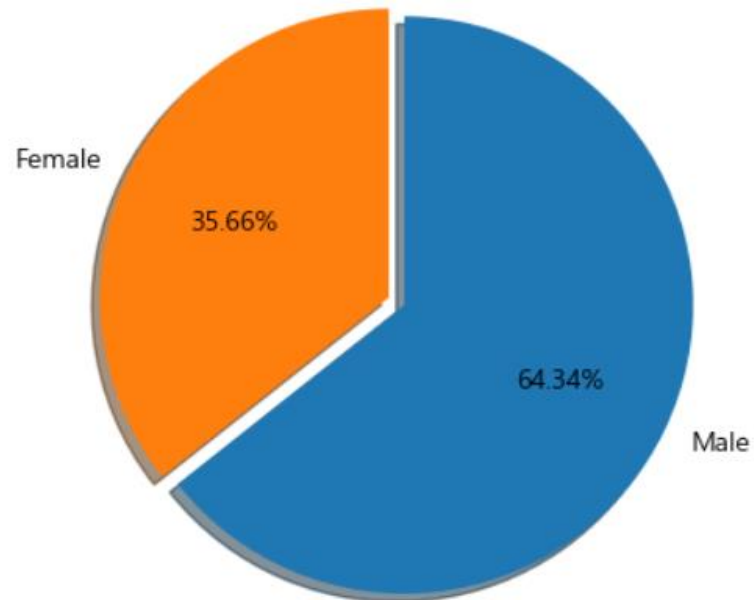
• 다양한 그래프

> 막대 그래프

- startangle, counterclock, explode, shadow

```
plt.pie(s_tips.values, labels = s_tips.index, autopct = '%.2f%%',  
        startangle = 90, counterclock = False,  
        explode = [0.03, 0.03], shadow = True)
```

Out :



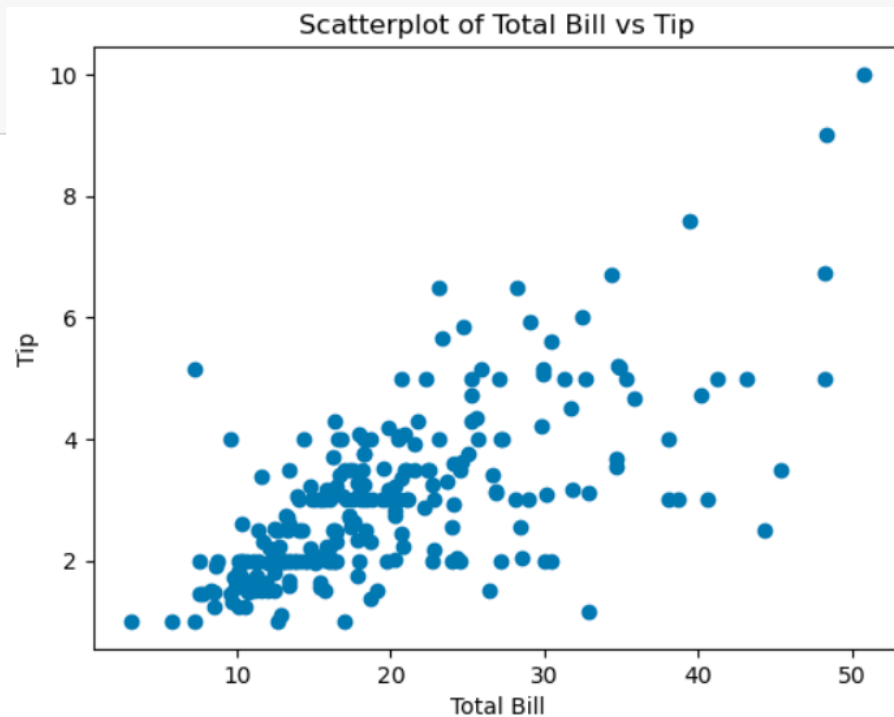
• 다양한 그래프

> 산점도

- 데이터의 분포를 표현

```
plt.scatter(tips['total_bill'], tips['tip'])  
plt.title('Scatterplot of Total Bill vs Tip')  
plt.xlabel('Total Bill')  
plt.ylabel('Tip')
```

Out :



• 다양한 그래프

> 3개 이상의 변수를 사용한 그래프

- x축, y축을 제외한 색상, 크기 등의 차이로 데이터를 구분

```
def recode_gender(gender):  
    if gender == 'Female':  
        return 'red'  
    else:  
        return 'blue'  
tips['color'] = tips['sex'].apply(recode_gender)  
tips.head()
```

Out :

	total_bill	tip	sex	smoker	day	time	size	color
0	16.99	1.01	Female	No	Sun	Dinner	2	red
1	10.34	1.66	Male	No	Sun	Dinner	3	blue
2	21.01	3.50	Male	No	Sun	Dinner	3	blue
3	23.68	3.31	Male	No	Sun	Dinner	2	blue
4	24.59	3.61	Female	No	Sun	Dinner	4	red

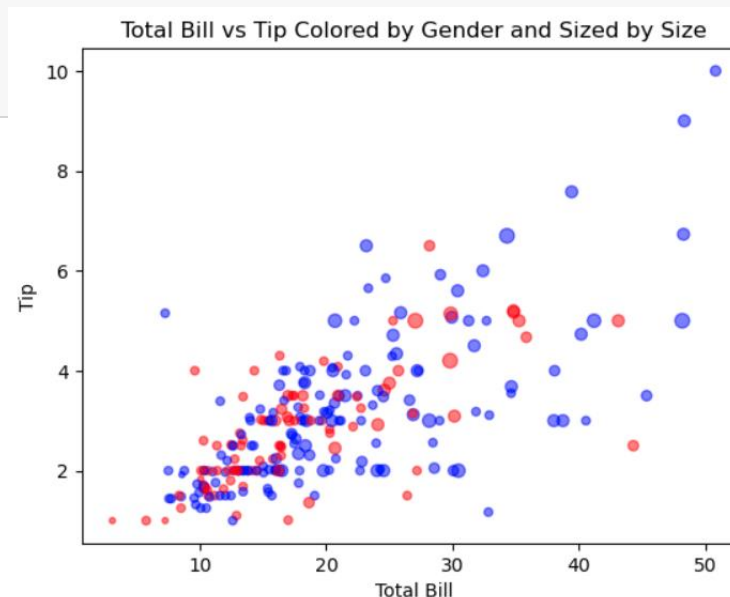
• 다양한 그래프

> 3개 이상의 변수를 사용한 그래프

- x축, y축을 제외한 색상, 크기 등의 차이로 데이터를 구분

```
plt.scatter(x = tips['total_bill'], y = tips['tip'], s = tips['size'] * 10,  
            c = tips['color'], alpha = 0.5)  
plt.title('Total Bill vs Tip Colored by Gender and Sized by Size')  
plt.xlabel('Total Bill')  
plt.ylabel('Tip')
```

Out :



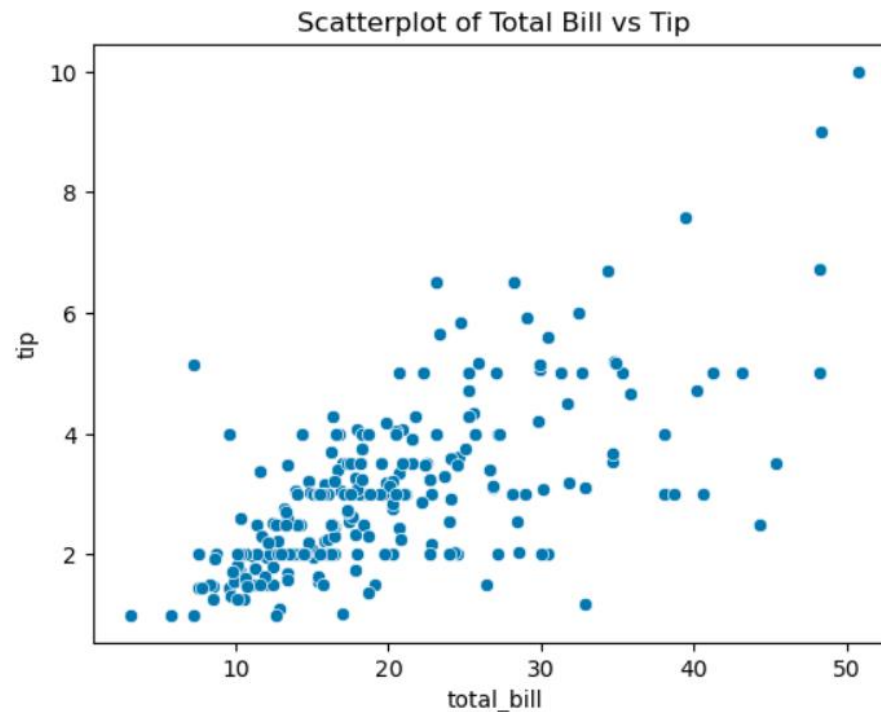
- Seaborn 라이브러리

- > 기본 그래프

- 산점도 scatterplot

```
sns.scatterplot(x = 'total_bill', y='tip', data=tips)
plt.title('Scatterplot of Total Bill vs Tip')
```

Out :



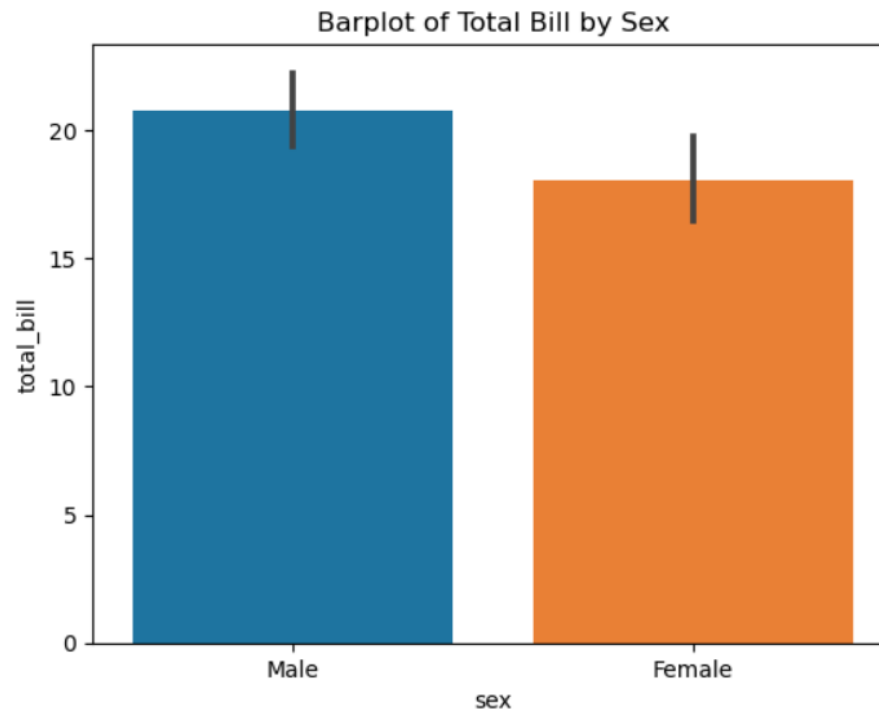
- Seaborn 라이브러리

- > 기본 그래프

- 막대 barplot, countplot

```
sns.barplot(data = tips, x = 'sex', y = 'total_bill')  
plt.title('Barplot of Total Bill by Sex')
```

Out :



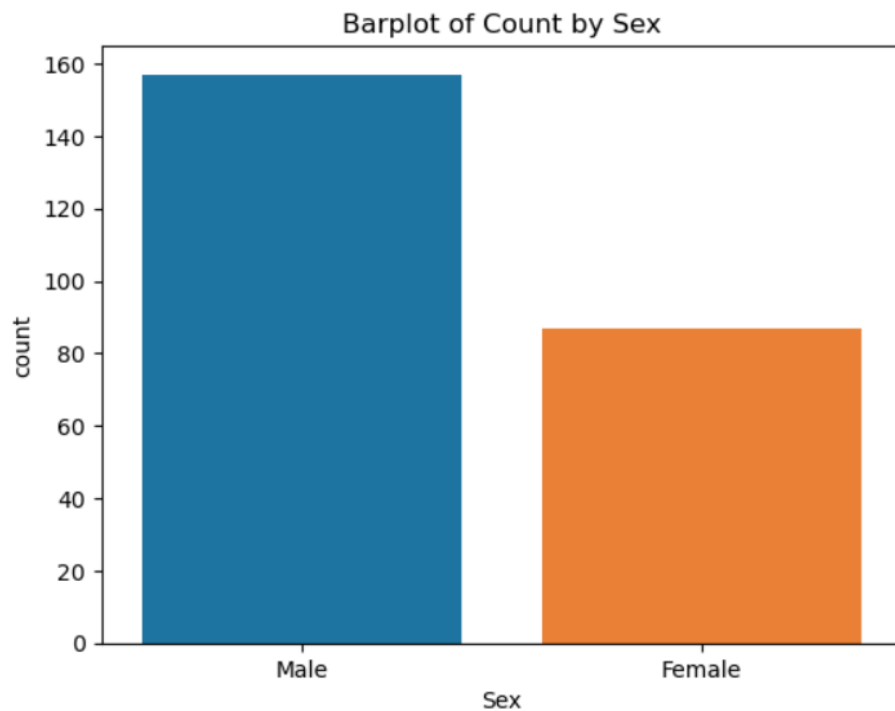
- Seaborn 라이브러리

- > 기본 그래프

- 막대 barplot, countplot

```
sns.countplot(data = tips, x = 'sex')  
plt.title('Barplot of Count by Sex')
```

Out :



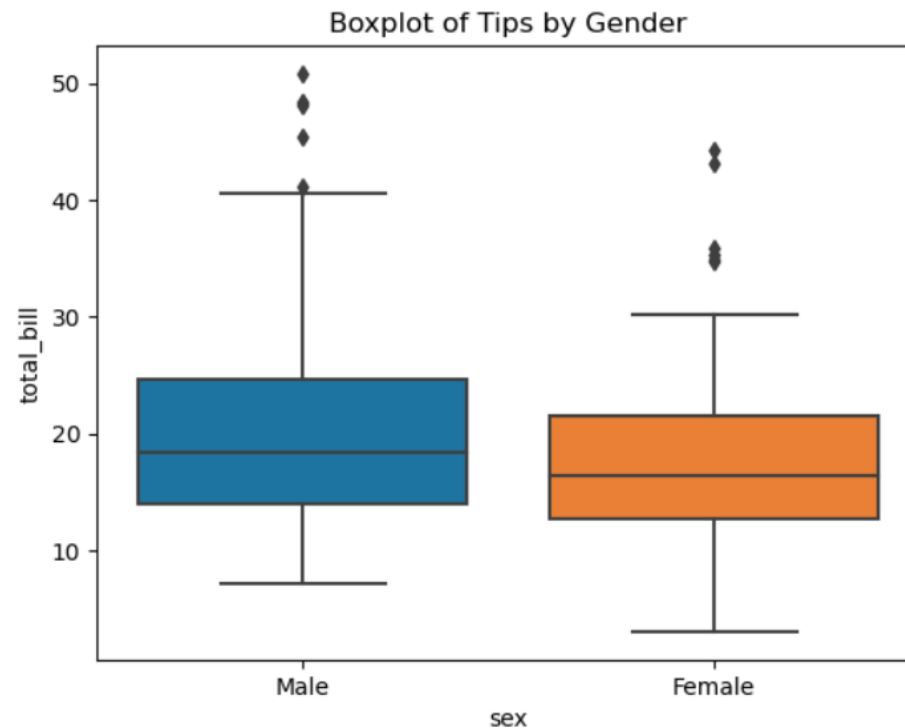
- Seaborn 라이브러리

- > 기본 그래프

- 박스 boxplot

```
sns.boxplot(x = 'sex', y = 'total_bill', data = tips)  
plt.title('Boxplot of Tips by Gender')
```

Out :



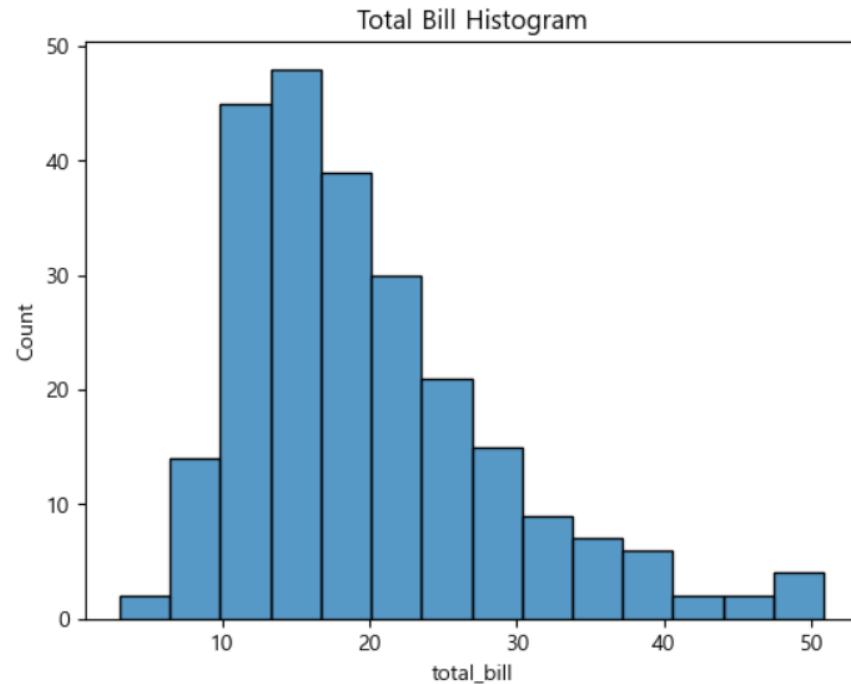
- Seaborn 라이브러리

- > 기본 그래프

- histplot

```
sns.histplot(tips['total_bill'])  
plt.title('Total Bill Histogram')
```

Out :



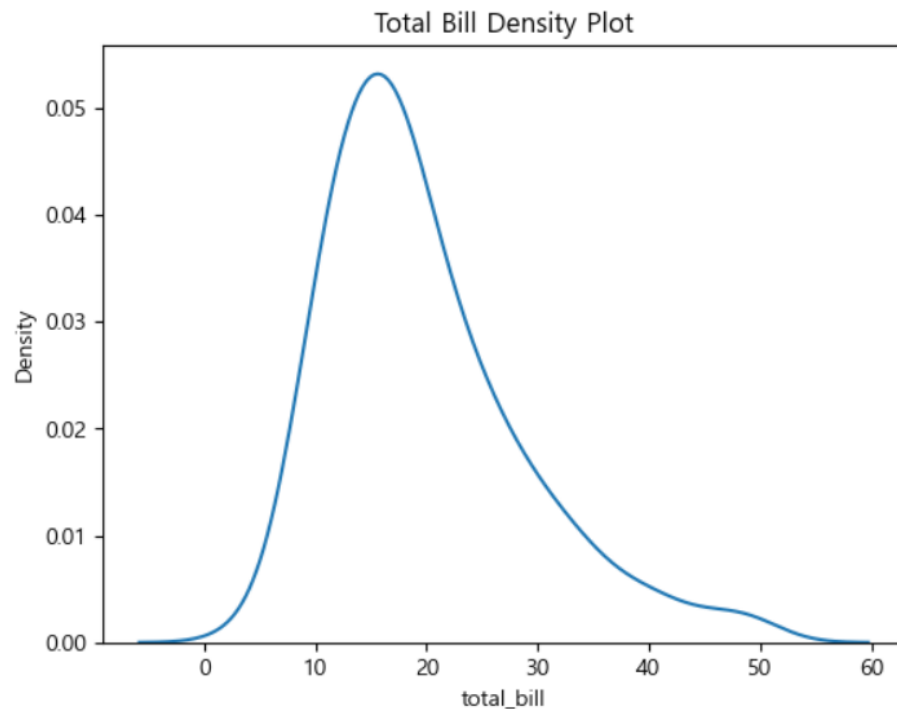
- Seaborn 라이브러리

> 새로운 그래프

- Density

```
sns.kdeplot(tips['total_bill']) # Kernel density estimation  
plt.title('Total Bill Density Plot')
```

Out :



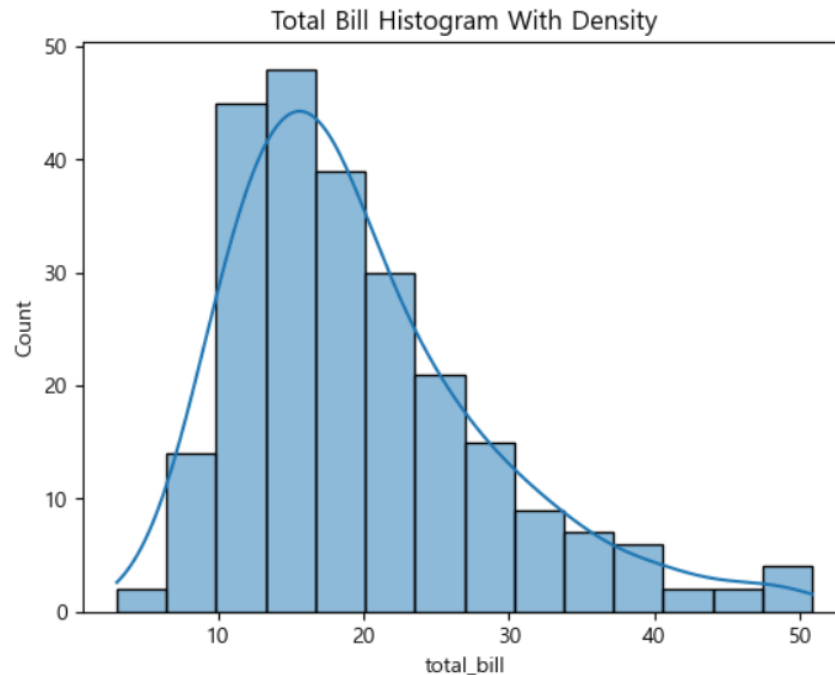
- Seaborn 라이브러리

- > 새로운 그래프

- histplot(Histogram + Density)

```
sns.histplot(tips['total_bill'], kde=True)  
plt.title('Total Bill Histogram With Density')
```

Out :



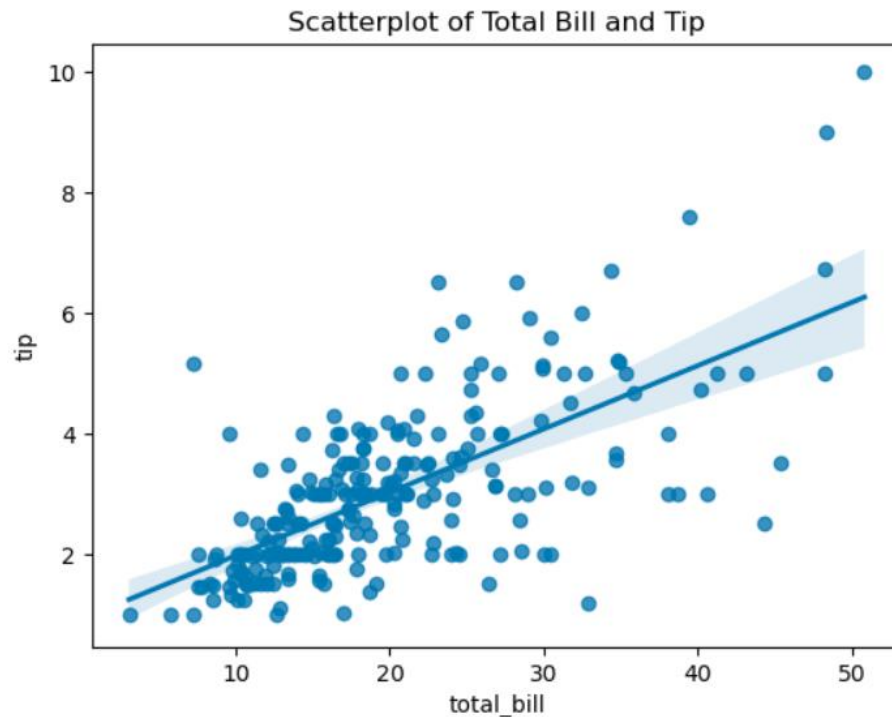
- Seaborn 라이브러리

- > 새로운 그래프

- regplot(Scatter + Regression)

```
sns.regplot(x = 'total_bill', y = 'tip', data = tips)  
plt.title('Scatterplot of Total Bill and Tip')
```

Out :



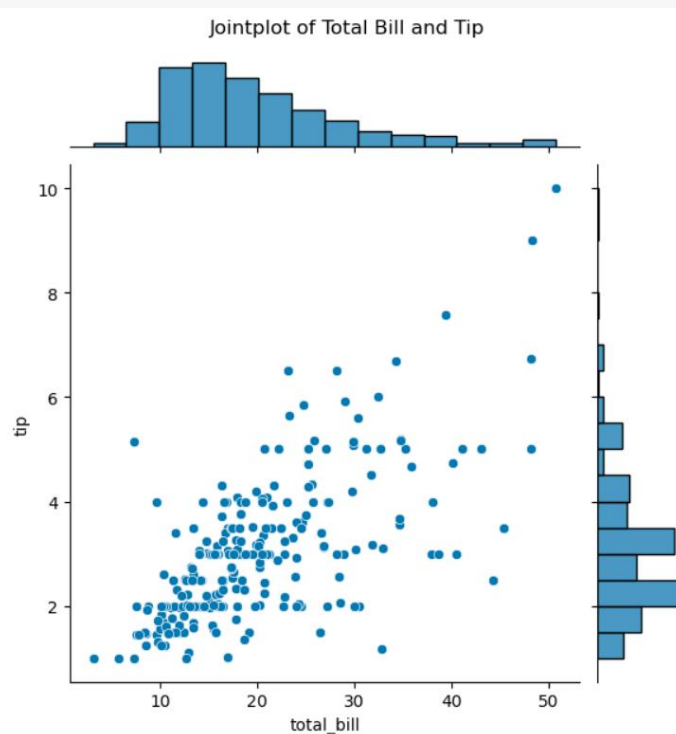
- Seaborn 라이브러리

- > 새로운 그래프

- jointplot(Scatter + Histogram)

```
sns.jointplot(x = 'total_bill', y = 'tip', data = tips)
plt.suptitle('Jointplot of Total Bill and Tip')
plt.tight_layout()
```

Out :



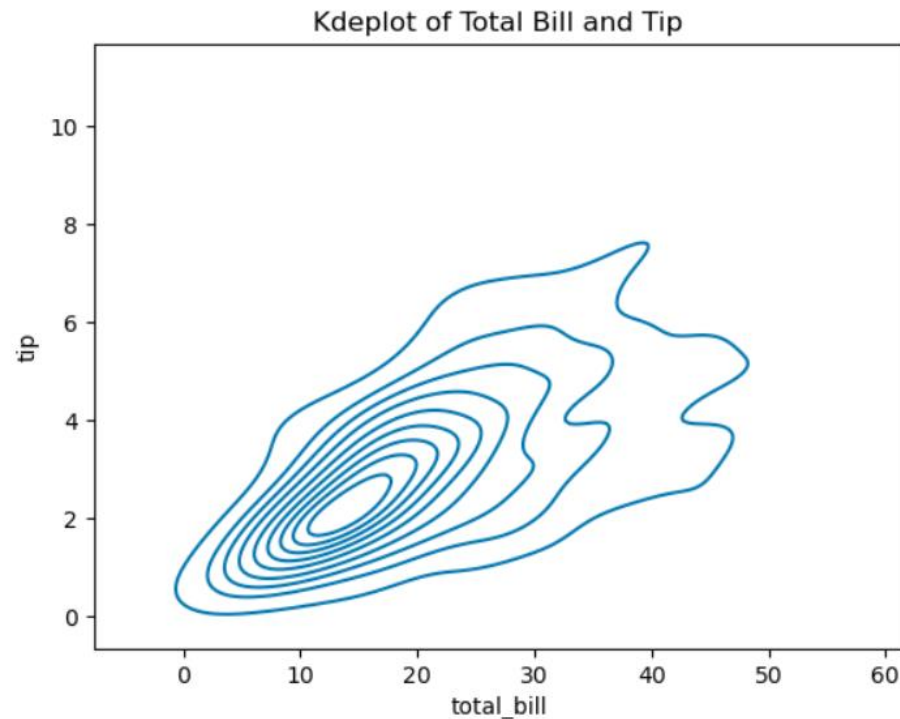
- Seaborn 라이브러리

> 새로운 그래프

- kdeplot

```
sns.kdeplot(x = 'total_bill', y = 'tip', data = tips)  
plt.title('Kdeplot of Total Bill and Tip')
```

Out :



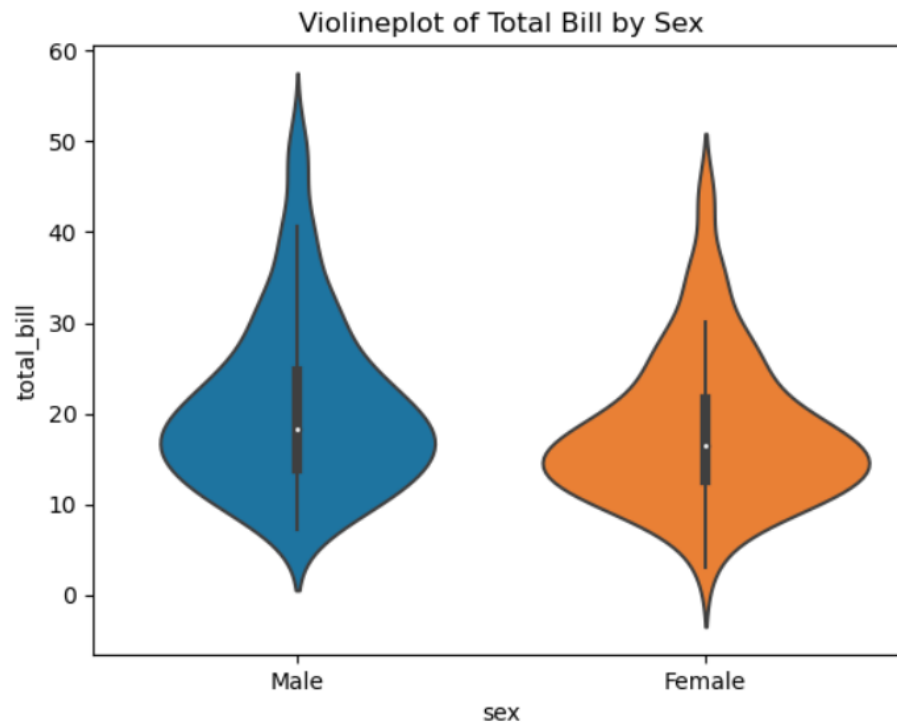
- Seaborn 라이브러리

> 새로운 그래프

- violinplot

```
sns.violinplot(x = 'sex', y = 'total_bill', data = tips)  
plt.title('Violineplot of Total Bill by Sex')
```

Out :



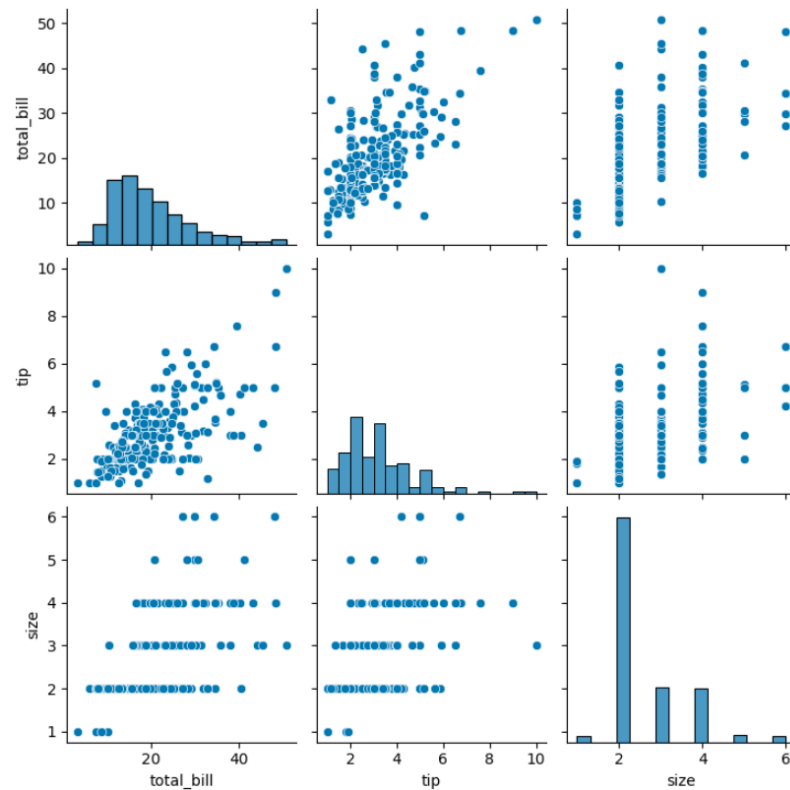
- Seaborn 라이브러리

- > 새로운 그래프

- pairplot - 각 컬럼(숫자) 간의 히스토그램, 산점도 그래프

```
sns.pairplot(tips)
```

Out :



- Seaborn 라이브러리

- > 새로운 그래프

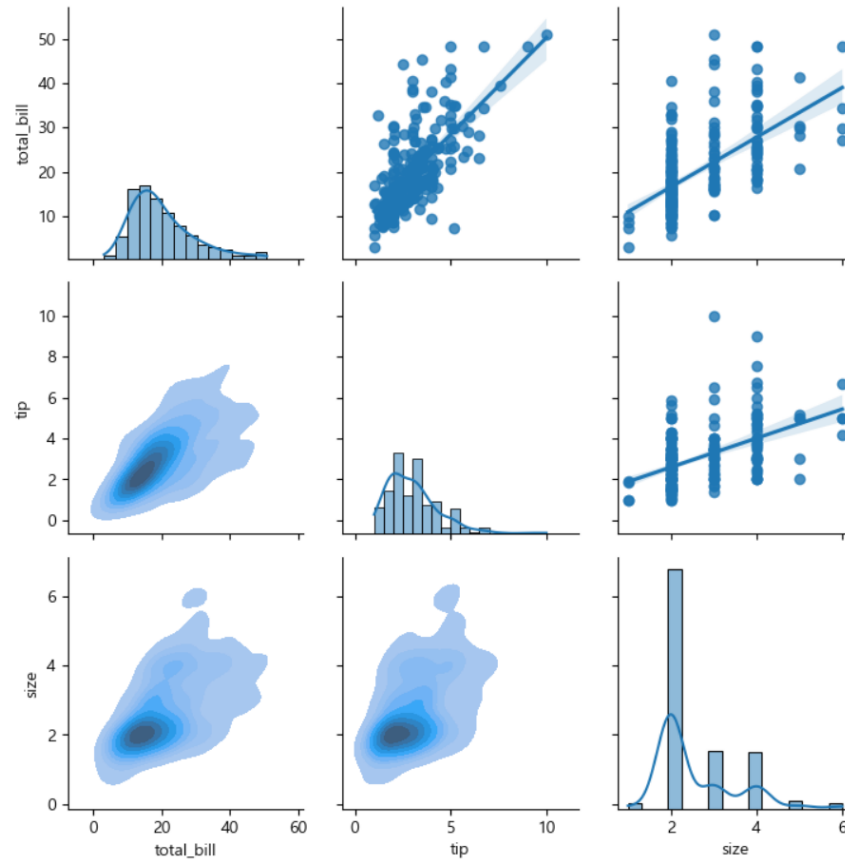
- pairplot - 각 컬럼(숫자) 간의 히스토그램, 산점도 그래프

```
pair_grid = sns.PairGrid(tips)
pair_grid = pair_grid.map_upper(sns.regplot)
pair_grid = pair_grid.map_lower(sns.kdeplot, fill = True)
pair_grid = pair_grid.map_diag(sns.histplot, kde=True)
```

- Seaborn 라이브러리

- > 새로운 그래프

- pairplot - 각 컬럼(숫자) 간의 히스토그램, 산점도 그래프



- Seaborn 라이브러리

- > 새로운 그래프

- pairplot - 각 컬럼(숫자) 간의 히스토그램, 산점도 그래프

```
sns.pairplot(tips, hue='sex')
```

Out :

